

Bachelorarbeit

**Predicting Open-Source Dependency Inactivity Risk for Secure Software
Supply Chains**

zur Erlangung des akademischen Grades
'Bachelor of Science' B.Sc.
im Studiengang 'Informatik'

vorgelegt dem Fachbereich Informatik und Wirtschaftsinformatik der
Provadis School of International Management and Technology
von

Eric Krause
Matrikelnummer: D720
BIN-T23-F-4
eric.krause@telekom.de

Erstgutachter: Prof. Dr. Lamya Abdullah

Zweitgutachter: Dr. Peter Manshausen

Ende der Bearbeitungsfrist: 29.07.2026

Abstract

Modern software depends on open-source components, but downstream projects face maintenance and security exposure when an upstream repository becomes inactive. This thesis investigates whether public repository health and maintenance indicators predict inactivity within twelve months and support dependency triage. Following a Design Science Research-oriented approach, it develops OSS Risk Radar. The artifact reconstructs a leakage-controlled historical dataset from GH Archive and public package metadata, computes observation-time activity and maintenance features, and trains calibrated inactivity-risk models.

The foundation dataset contains 5,000 repositories and 50,000 quarterly snapshots. Full-history XGBoost reaches held-out ROC-AUC 0.958, the probability that an inactive repository ranks above an active one, with Brier score 0.078. A repository-disjoint split reaches 0.950. On rows where commit recency is defined, that trivial rule reaches 0.957, compared with 0.974 for XGBoost on the same rows. Among repositories still active at the observation date, XGBoost reaches 0.905 against 0.844 for the best trivial rule. Most aggregate discrimination is therefore available from recency, while the learned model adds more for early decline.

Security-practice signals remain separate triage context and do not enter the inactivity model. A practical demonstrator applies the staged models per repository to a real project's dependencies and produces a ranked view with evidence support. It demonstrates technical integration, not improved analyst decisions. Repository inactivity remains an operational proxy for elevated maintenance risk, not proof of abandonment or software insecurity.

Contents

Abstract	ii
List of Figures	vi
Glossary	vii
List of Tables	x
List of Abbreviations	xii
1 Introduction	1
1.1 Motivation and Problem Statement	1
1.2 Research Question and Objectives	2
1.3 Scope and Assumptions	3
1.4 Contributions	3
1.5 Structure of the Thesis	4
2 Background	5
2.1 Software Supply-Chain Security	5
2.2 Open-Source Sustainability and Project Health	5
2.3 Metric Pitfalls and Limitations	6
2.4 Machine Learning Basics for Tabular Risk Scoring	6
3 Related Work	8
3.1 OSS Inactivity and Abandonment Operationalizations	8
3.2 Repository Health Metrics and Dashboards	9
3.3 Security-Practice Indicators	9
3.4 Responsiveness Measurement Caveats	9
3.5 Research Gap	10
4 Methodology	11
4.1 Research Design	11
4.2 Iterative Artifact Development	12
4.3 Definitions and Prediction Target	12
4.4 Data Sources	14
4.5 Dataset Construction Workflow	16
4.6 Observation Window and Historical Coverage	16
4.7 Feature Engineering	18

4.8	Full-History and Cold-Start Prediction Regimes	19
4.9	Leakage Prevention	20
4.10	Model Training	20
4.11	Evaluation Strategy	21
4.12	Methodological Limitations	22
5	Implementation	23
5.1	System Architecture Overview	23
5.2	Historical Dataset Builder	23
5.3	Model Training and Evaluation Instrumentation	26
5.4	Reproducibility of the Evaluated Artifact	27
6	Results and Discussion	28
6.1	Predictive Performance	28
6.1.1	Comparison Against Non-Learned Baselines	29
6.1.2	Early Warning: Removing the Already-Inactive Repositories	29
6.1.3	Precision-Recall Evaluation of the Early-Warning Subset	30
6.2	Calibration and Threshold Selection	31
6.3	Feature Importance and Error Analysis	31
6.3.1	Feature Attribution	31
6.3.2	Robustness to Repository Recurrence	33
6.3.3	Slice Evaluation	33
6.3.4	Error Analysis	33
6.4	Robustness to the Label Definition	34
7	Practical Demonstrator	35
7.1	Example Repository Set	35
7.2	Risk Ranking Output and Interpretation Guidance	36
8	Threats to Validity	39
8.1	Construct Validity	39
8.2	Internal Validity	39
8.3	External Validity	40
8.4	Conclusion Validity	41
9	Conclusion and Outlook	42
9.1	Outlook and Future Work	42
	Bibliography	I
	Declaration on the Use of Artificial Intelligence	V

A Appendix	XIII
A.1 Code and Data Availability	XIII
A.2 Observable Health Indicator Categories	XIII
A.3 Full-History Feature Reference	XIII
A.4 Foundation Seed Composition	XIX
A.5 Undefined Feature Handling	XXI
A.6 System Components and Technology Selection	XXIII
A.7 Dataset Builder and Training Pipeline	XXV
A.7.1 Dataset-Builder Modules	XXV
A.7.2 Dataset-Quality Reporting	XXVI
A.7.3 Feature-Set Refinement After the Formative Review	XXVI
A.7.4 Model Configurations and Hyperparameters	XXVII
A.7.5 Artifact Serialization	XXVII
A.7.6 Training Orchestration	XXVIII
A.7.7 Artifact Staging and Promotion Gate	XXVIII
A.8 Runtime Services and Deployment	XXIX
A.8.1 Scoring Service	XXIX
A.8.2 Backend Orchestration Service	XXX
A.8.3 Analyst Frontend	XXXI
A.8.4 Container and Cluster Deployment	XXXI
A.9 Evaluation Instruments	XXXII
A.9.1 Subgroup Definitions for the Slice Evaluation	XXXII
A.9.2 Mechanics of the Repository-Disjoint Split	XXXIII
A.10 Dataset Construction Steps	XXXIV
A.11 Evaluation Metric Definitions	XXXIV
A.12 Reproducibility	XXXVI
A.13 Label-Definition Sensitivity	XXXVIII
A.14 Full Demonstrator Ranking	XXXIX
A.15 Per-Subgroup Metrics of the Slice Evaluation	XLI
A.16 Ablation: The Human-Actor Filter	XLI
Declaration of Authorship	XLIII

List of Figures

4.1	Temporal structure of a repository observation snapshot	13
4.2	Dataset construction pipeline and its leakage boundary	16
5.1	System architecture of OSS Risk Radar	24
6.1	Standardized logistic-regression coefficients	32
7.1	Risk-bucket and action-level distribution of the demonstrator run	36

Glossary

This glossary explains the recurring technical terms of the thesis.¹ Terms from machine-learning practice are defined in the sense in which they are standard in that literature; where this thesis defines a term of its own, the entry says so. The formal equations for the evaluation metrics are given in Appendix A.11.

Term	Definition
Ablation	An experiment that removes or disables one component of a system and retrains it, so that the difference in results measures what that component actually contributed. Used here for the human-actor filter (Appendix A.16) and for feature availability (Section 4.8).
Average precision	Area under the precision-recall curve: a ranking measure that, unlike ROC-AUC, ignores correctly ranked negatives and so is not flattered by a large negative class. Its no-skill reference is the positive rate of the evaluated set, not 0.5, so it must be read alongside that rate (Section 6.1.3).
Brier score	A measure of how accurate predicted <i>probabilities</i> are: the mean squared difference between the predicted probability and the observed outcome. Lower is better.
Calibration	Whether predicted probabilities match reality in the long run — if the model says 0.8 for a group of repositories, roughly 80% of them should in fact become inactive. Distinct from discrimination: a model can rank correctly and still be badly calibrated.
Cold-start	The standard machine-learning term for having to make a prediction about an entity for which no history is available. Here it names the model that uses only the 17 current-snapshot features, and applies to any repository absent from the precomputed feature cache.
Discrimination	Whether the model puts the repositories that became inactive above those that did not, irrespective of the absolute probabilities. Measured by ROC-AUC.

¹Claude Opus 5/3 — see the Declaration on the Use of Artificial Intelligence, p. V.

Term	Definition
Expected calibration error (ECE)	A single number summarizing miscalibration: predictions are grouped into probability bins and the average gap between each bin’s predicted probability and its observed outcome frequency is reported.
F1 score	The harmonic mean of precision and recall at a chosen decision threshold, i.e. a single number balancing how many flagged repositories were really inactive against how many inactive repositories were flagged. Unlike ROC-AUC it depends on the threshold.
Full-history	The counterpart to cold-start: the model that additionally uses the 43 features reconstructed from the event history before the observation date.
Held-out test set	The rows set aside before training and used neither for fitting the model nor for calibrating its probabilities, so that the reported metrics describe performance on data the model has never seen.
Human-actor filter	A filter of this thesis that excludes bot accounts from commit- and contributor-derived signals and from the label, so that automated activity cannot be mistaken for maintenance (Section 4.7).
Inactivity label	The prediction target: 1 if the repository is <i>not</i> maintained during the twelve months after the observation date, 0 otherwise (Section 4.3).
Leakage	The error of letting information that would not have been available at prediction time influence the model, which inflates measured performance. Prevented here by the temporal boundary at the observation date (Section 4.9).
Observation snapshot	One row of the dataset: a single repository described as it appeared at one observation date, together with the label derived from the following twelve months.
Prediction regime	The term used in this thesis for which of the two nested feature sets a model was trained on — full-history or cold-start.

Term	Definition
Reconstructed proxy	A feature that measures the same underlying construct as an established metric but is computed by this thesis from GH Archive event data under its own rules, and is therefore not that metric. The responsiveness features are reconstructed proxies for, not implementations of, the corresponding CHAOSS metrics (Section 3.4).
Right-censored	A value that is known only as a lower bound. If a repository has no observed commit, “days since last commit” has no exact value but is at least the length of the observable record, and is encoded at that bound rather than as zero (Section 4.7).
ROC-AUC	Area under the receiver operating characteristic curve: the probability that a randomly chosen inactive repository is scored above a randomly chosen active one. 0.5 is a coin flip, 1.0 a perfect ranking.
Slice evaluation	Re-computing the metrics separately within meaningful subgroups of the test set (for example by popularity, or restricted to repositories still active at the observation date), so that strong performance on a large subgroup cannot hide weak performance on a smaller one. The word <i>slice</i> is used in this thesis only for such a subgroup; the three data partitions are called the training, validation, and test sets.
Two-of-four maintenance rule	This thesis’s own operational definition of maintenance: a repository counts as maintained in the twelve-month window if at least two of four activity checks are satisfied (Section 4.3). It is a design choice of this work, not an established external standard, and its sensitivity is tested in Section 6.4.

List of Tables

6.1	Held-out test metrics by model family and feature regime, with the two trivial baselines. Higher is better for ROC-AUC, F1, and quality; lower is better for Brier and ECE. The best value within each regime is shown in bold. *The recency baseline is defined on the 4,556 test rows that have an observable commit history; see the text for the like-for-like comparison.	28
6.2	Held-out ROC-AUC by feature-set size, on identical row sets.	30
6.3	Held-out average precision on identical row sets, with 95% bootstrap intervals. The no-skill column is the positive rate, which a random ranking attains.	30
7.1	Flagged dependency repositories of ArchipelagoMW/Archipelago (the two <code>replace_candidate</code> items and the four <code>review</code> items), with the single full-history repository shown for contrast. Risk is the 0–100 inactivity-risk score, and “Supp.” is the per-prediction evidence support. Sec. is the 0–100 security-posture score derived from the repository’s OpenSSF Scorecard record; no part of the inactivity model reads it (Section 4.1). It is shown as — where no Scorecard record was available. “Regime” is full-history (FH) or cold-start (CS). The complete 30-repository ranking is in Appendix A.14.	37
9.1	Use of generative AI systems in the preparation of this thesis and the software artifact, listing the system and version, the opening prompt, and how and where the results were used.	V
A.1	Categories of observable health indicators and their expected relevance	XIV
A.2	Implemented full-history feature groups	XV
A.3	Historical feature reference	XV
A.4	Current-snapshot feature reference: the 17 features obtainable without event history. These form the complete cold-start vector and are also included in the full-history vector.	XX
A.5	Foundation seed buckets: target frame versus realized seed (5,000 repositories).	XX
A.6	Implemented system components	XXIV
A.7	Dataset builder modules	XXV
A.8	Historical dataset construction workflow	XXXIV

A.9	Sensitivity of the held-out evaluation to the maintenance-rule threshold. “Inactive rate” is the positive-class share of the test set. “XGBoost (all)” is the full-history model on all 5,000 test rows; the remaining columns are on the 4,556 recency-defined rows, where model and baselines are scored on identical rows. “Margin” is the model there minus the stronger of the two trivial rules, which is the recency rule under the permissive variant and commit volume under the other two.	XXXIX
A.10	Complete ranked inactivity-risk view for the dependency repositories of ArchipelagoMW/Archipelago, as produced by the model. Risk is the 0–100 inactivity-risk score; “Supp.” is the per-prediction evidence support; “Sec.” is the parallel 0–100 security-posture score derived from the repository’s OpenSSF Scorecard record, which is not an input to the inactivity model (Section 4.1) and is shown as — for the three repositories with no available Scorecard record; “Regime” is full-history (FH) or cold-start (CS).	XL
A.11	Held-out ROC-AUC and Brier score of <code>xgboost-full-history</code> within subgroups of the test set. “Inactive rate” is the positive rate in the subgroup.	XLI
A.12	Cross-test of the human-actor filter: training on bot-contaminated features versus clean features, both evaluated against the identical clean (human-filtered) label on the test set. A lower ROC-AUC or higher Brier for the contaminated features means the filter helps.	XLII

List of Abbreviations

API	Application Programming Interface
CHAOSS	Community Health Analytics in Open Source Software
CS	Cold-Start (scoring regime)
CVSS	Common Vulnerability Scoring System
ECE	Expected Calibration Error
FH	Full-History (scoring regime)
ML	Machine Learning
NIST	National Institute of Standards and Technology
OSS	Open Source Software
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
SLSA	Supply-chain Levels for Software Artifacts
SPDX	Software Package Data Exchange
SSDF	Secure Software Development Framework
XGBoost	Extreme Gradient Boosting

1 Introduction

This chapter motivates repository maintenance as a dependency-security concern, states the research question, and defines the scope and contributions of the thesis.

1.1 Motivation and Problem Statement

Modern software systems combine first-party code with third-party and open-source dependencies. Studies across Java, Python, Ruby, and npm describe applications that depend substantially on externally maintained packages, including transitive dependencies [1], [2]. Here, open-source software means software distributed under a license that complies with the Open Source Definition [3]; Section 2.1 gives the licensing criteria.

Depending on external components transfers part of the maintenance burden to people outside the downstream project. If maintainers no longer address a disclosed vulnerability, the downstream project must fork the component, vendor a private patch, or migrate instead of applying a normal version update. Until then, the dependency remains exposed without an upstream fix. Secure-development guidance therefore treats dependency upkeep as a supply-chain control. The NIST Secure Software Development Framework includes acquiring and maintaining well-secured third-party and open-source components [4]. SLSA addresses artifact and build-process integrity and provenance [5].¹

Miller et al. found that 15% of widely used npm packages became abandoned within six years. Only 18% of the exposed downstream projects removed the abandoned dependency [2]. Their earlier interview study found that affected developers often lack clear response strategies [6]. Delayed maintenance also prolongs security exposure. Across dependencies in 450 Java, Python, and Ruby projects, publicly known vulnerabilities persisted throughout a one-year observation period at mean rates of 78.4%, 97.7%, and 66.4%, respectively. Resolved vulnerabilities took three to five months [1].²

Several incidents illustrate the consequences. The 2016 `left-pad` unpublishing disrupted many direct and transitive dependants [7]. The 2017 Equifax breach exploited an Apache Struts vulnerability whose available patch had not been applied, and Equifax later agreed to a settlement of up to \$700 million [8], [9]. The XZ Utils backdoor made maintainer trust an attack surface through a social-engineering takeover [10].

An inactive repository is not necessarily insecure or abandoned; a project may be stable, paused,

¹Claude Opus 5/1 — see the Declaration on the Use of Artificial Intelligence, p. V.

²Claude Opus 5/10 — see the Declaration on the Use of Artificial Intelligence, p. V.

or maintained through channels not visible in public data. The examples nevertheless show how dependency maintenance can affect downstream security exposure. This thesis uses repository inactivity as an operational proxy for elevated maintenance risk and examines whether public signals can estimate that risk over a 12-month horizon.

1.2 Research Question and Objectives

The central research question of this thesis is:

To what extent can a machine-learning-based scoring tool, using only publicly observable repository health and maintenance indicators, predict 12-month OSS dependency inactivity, and how can this inactivity-risk score be combined with parallel security-practice signals to support risk-based dependency triage in secure dependency management?

To answer this research question, the thesis pursues the following objectives:

1. Define an operational inactivity label for open-source repositories within a 12-month prediction horizon.
2. Construct a reproducible dataset from publicly observable repository metadata and dependency-related sources.
3. Engineer observation-time activity and maintenance features for the inactivity model, and surface selected security-practice signals as parallel triage context rather than as inputs to the inactivity prediction.
4. Train and compare simple baselines and machine-learning models for inactivity-risk prediction.
5. Evaluate the models using discrimination and calibration metrics that are suitable for risk scoring.
6. Demonstrate how the trained model can be applied per repository to a set of dependency repositories to support risk-based triage.

The research follows a Design Science Research-oriented approach and develops an inactivity-risk scoring workflow for open-source dependencies [11], [12]. The empirical evaluation measures discrimination and calibration. The *demonstrator* is a worked application of the artifact to a real case: Chapter 7 applies the trained model to the dependency repositories of an existing open-source project. It shows integration into dependency monitoring and behavior outside the sampled evaluation set, while Chapter 6 provides the measurement of predictive accuracy. This

distinction defines the role of the *demonstrator* throughout the thesis.³

1.3 Scope and Assumptions

The scope covers dependencies with publicly observable source repositories and package meta-data that can be associated with an open-source license through GitHub’s SPDX-based license information [13]. It excludes private repositories, closed-source vendor components, and packages that cannot be mapped to a source repository (Section 4.4).

The thesis covers only projects developed on *GitHub*. Both training data and observation-time features are reconstructed from its public event stream (Section 4.4). Projects on GitLab, Codeberg, self-hosted forges, or mailing-list workflows such as that of the Linux kernel are outside the scope. This non-random exclusion limits generalization to open-source software as a whole (Section 8.3). A GitHub mirror can also appear inactive although development continues elsewhere.⁴

The prediction target is repository inactivity within $[t, t + 12 \text{ months}]$. The label uses four activity checks: human commit activity, active contributors, releases or package publications, and merged pull requests. At least two must hold for the repository to count as maintained (Section 4.3), building on revision activity as a survival signal [14]. Merged pull requests serve as a proxy for maintainer interaction. Issue activity and responsiveness are predictors only and require caution because bots can distort response measures [15].

Public signals provide incomplete evidence about future inactivity, and health indicators can have different meanings across projects [16]. The model output therefore supports triage but does not automate the decision.

1.4 Contributions

The thesis makes three contributions:

1. an operational 12-month inactivity-risk label and a reproducible workflow for constructing it;
2. an empirical evaluation of inactivity prediction from public repository health and maintenance indicators, benchmarked against simple baselines on discrimination and probability calibration, with security-practice signals kept outside the inactivity model as complementary triage context;

³Claude Opus 5/1 — see the Declaration on the Use of Artificial Intelligence, p. V.

⁴Claude Opus 5/1 — see the Declaration on the Use of Artificial Intelligence, p. V.

3. a per-repository demonstrator that turns the score into a ranked triage view.

The score provides a reproducible way to prioritize dependencies for expert review.

1.5 Structure of the Thesis

Chapters 2 and 3 provide the conceptual and empirical background. Chapters 4 and 5 describe the method and its implementation. Chapter 6 evaluates the models, and Chapter 7 applies them in a ranked triage view. Chapters 8 and 9 discuss validity and future work.

2 Background

This chapter covers software supply-chain security, open-source project health, metric limitations, and the machine-learning concepts used for inactivity-risk scoring.

2.1 Software Supply-Chain Security

Software supply-chain security addresses insufficiently controlled or verifiable artifacts, build processes, dependencies, and distribution channels [4], [5]. The NIST Secure Software Development Framework includes the acquisition and maintenance of well-secured components [4]. SLSA defines integrity and provenance requirements for artifacts and build processes [5]. The NIST requirement motivates examining upstream maintenance in addition to dependency versions.

Open-source licenses permit use, modification, and redistribution under criteria specified by the Open Source Initiative [3]. These freedoms enable broad reuse, while downstream users depend on external maintainers and public infrastructure. Upstream activity is therefore relevant to downstream risk assessment. Inactivity-risk scoring estimates whether public repository signals warrant monitoring, replacement planning, or manual review; it does not assess a dependency's overall security.

The unit of analysis is one repository. Software-composition-analysis tooling provides the dependency inventory as an external input (Section 5.1).

2.2 Open-Source Sustainability and Project Health

Open-source sustainability describes whether a project can continue to provide value and maintain its software over time. Project health is commonly assessed through observable indicators such as activity, responsiveness, contributor participation, and release behavior [17], for which the CHAOSS project provides standardized metrics models [18].

Repository activity includes commits, releases, tags, issues, pull requests, and contributors observed through public platforms and package metadata. Prior research uses revision activity and project attributes to study open-source project survival [14]. These signals support the 12-month inactivity prediction developed here.

OpenSSF Scorecard supplies a second category of signals through automated security-related checks that describe repository configuration and security posture [19]. Section 3.3 explains why

these signals remain separate from the inactivity model.

Project health remains broader than any single metric [17]. The inactivity-risk score is therefore a limited operational signal for dependency triage.

Appendix A.2 (Table A.1) groups the signals into commit activity, contributor activity, issue and pull-request activity, release activity, age and popularity, and security practice. Section 4.7 defines the model features.

2.3 Metric Pitfalls and Limitations

Repository metrics can be misleading without context [16]. Their meaning depends on project type, maturity, governance, and maintenance style.

A low commit count can indicate reduced maintenance or a stable project needing few changes. A high issue count can represent an active community or an unresolved backlog. Context-free indicators are therefore insufficient on their own [16], [20]. The model combines multiple signals and evaluates them empirically.

Automated replies can distort time-to-first-response in pull-request workflows [15]. Human and automated activity should therefore be separated where possible (Section 3.4). The commit and contributor signals use a human-actor filter (Appendix A.3), whose effect is measured in Appendix A.16. The responsiveness features remain unfiltered reconstructed proxies.

Public repository data may omit development on other platforms, releases outside the observed repository, private maintenance channels, or ambiguous package-to-repository mappings [20]. The results cover only publicly observable activity.

The inactivity label consequently records the absence of selected public activity signals within a defined horizon. It is a measurable proxy for elevated maintenance risk, not a declaration of project death or abandonment.

2.4 Machine Learning Basics for Tabular Risk Scoring

The prediction task is binary classification: at observation time t the model estimates the probability that a repository becomes inactive in $[t, t + 12 \text{ months}]$ from features derived from public metadata and activity. Because inactivity is closely related to recent activity, a recency baseline — ranking repositories by the time since their last observed commit or release — serves as the reference point.

The recency baseline solves the same forward-looking task as the model. Both use only repository

state at t and are scored against $(t, t + 12 \text{ months}]$. The baseline extrapolates one variable: a repository silent for a long time is predicted to remain silent. The learned model combines many variables. This is a strict like-for-like comparison because the label is activity-derived and prolonged silence predicts future silence. Any improvement over the baseline cannot be obtained from the last commit date alone.¹

Logistic regression serves as an interpretable probabilistic baseline, while tree-based models such as gradient boosting capture the non-linear feature interactions common in tabular repository metadata [21], [22], [23].

The evaluation measures *discrimination* and *calibration*. Discrimination asks whether repositories that become inactive rank above those that remain active. Calibration asks whether a predicted probability such as 0.8 corresponds to an observed inactivity frequency of roughly 80%. A model can perform differently on the two criteria, so both are reported.

Discrimination is measured by **ROC-AUC**, the area under the receiver operating characteristic curve. It is the probability that a randomly chosen inactive repository receives a higher score than a randomly chosen active one. A value of 0.5 corresponds to random ordering; 1.0 ranks every inactive repository above every active one. ROC-AUC depends on score *order* and requires no decision threshold, which suits a ranked triage view. Under strong class imbalance it can appear optimistic, and the area under the precision-recall curve is more informative when the negative class dominates [24]. That condition does not hold for the full test set, whose majority class is inactive (Section 4.1.1), but it does hold for the subset of repositories still active at the observation date, for which **average precision** is therefore reported alongside ROC-AUC (Section 6.1.3).

Calibration is measured by the **Brier score** and the **expected calibration error (ECE)**. The Brier score is the mean squared difference between the predicted probability and the outcome, coded as 1 for inactive and 0 for active. Lower values are better. Predicting the base rate $\bar{p}$ for every repository yields $\bar{p}(1 - \bar{p})$, so a lower score indicates skill beyond that reference.

ECE groups predictions into probability bins and averages the absolute difference between each bin's predicted probability and observed outcome frequency. It expresses miscalibration in the units of the score. Calibration is required to interpret a score as a probability, for example as a one-in-four chance of inactivity within a year, when planning replacement [25]. Appendix A.11 defines these metrics, log loss, precision, recall, F1, and the composite quality score.

All metrics use a *held-out test set* excluded from model fitting and calibration. The split is time-aware, so later periods cannot influence model development for earlier ones. This matches the use of historical information to predict future inactivity.²

¹Claude Opus 5/2 — see the Declaration on the Use of Artificial Intelligence, p. V.

²Claude Opus 5/2 — see the Declaration on the Use of Artificial Intelligence, p. V.

3 Related Work

This chapter compares the inactivity-risk workflow with prior operationalizations of inactivity, project-health metrics, and security-practice indicators.

3.1 OSS Inactivity and Abandonment Operationalizations

Robinson et al. apply survival-style analysis to open-source Python projects and use revision activity as an observable signal for project survival [14]. Their results support reasoning about future project state from activity. A fitted hazard model can yield per-subject predictions, but their study reports population-level survival behavior. It does not evaluate fixed-horizon repository predictions against held-out future outcomes as done here.

Miller et al. find that 15% of widely used npm packages became abandoned within six years and that only 18% of exposed downstream projects removed the dependency [2].¹ Their interview study finds that developers often lack clear response strategies [6]. Both studies characterize abandonment retrospectively at ecosystem level. They do not predict a 12-month outcome for an individual repository from point-in-time features. Their findings motivate an early-warning triage signal.

Coelho et al. train a classifier that identifies unmaintained GitHub projects from activity features and validate its output with project developers [26]. Their follow-up tracks 2,927 active projects for one year and proposes a machine-learning metric for the *level* of maintenance activity. It reports that 16% became unmaintained [27]. At component level, OSSARA aggregates maintenance predictions to assess the abandonment risk of embedded open-source dependencies for continuous monitoring [28]. These studies show that maintenance status can be predicted and are the closest predictive comparisons for this thesis.

The present work differs in its operationalization:

- a fixed 12-month horizon evaluated from point-in-time observation snapshots reconstructed from GH Archive with an explicit temporal leakage boundary;
- a multi-signal inactivity label combining human commit, contributor, release, and merged-pull-request activity instead of a single revision-activity threshold or continuous activity index;
- probability *calibration* and comparison against strong trivial recency baselines in addition

¹Claude Opus 5/10 — see the Declaration on the Use of Artificial Intelligence, p. V.

to discrimination metrics;

- a separate analysis of the repositories still active at the observation time, where early warning is most decision-relevant;
- integration of the calibrated score into a per-repository dependency-triage demonstrator that keeps inactivity prediction and security posture explicitly separate.

Issue activity and responsiveness signals are kept as predictors only and are not part of the label.

3.2 Repository Health Metrics and Dashboards

The CHAOSS project provides standardized metrics models, including the Starter Project Health Metrics Model [18]. Goggins et al. argue that transparent project health requires structured measurement and interpretation instead of isolated indicators [17]. This vocabulary informs the feature engineering.

Dashboards and metrics models primarily describe indicators for human interpretation. They do not predict a future outcome. Yehudi et al. show that individual context-free indicators are insufficient for identifying sustainability across heterogeneous projects [16]. The model therefore combines observation-time signals and evaluates the combination empirically. Its output remains a limited operational proxy.

3.3 Security-Practice Indicators

OpenSSF Scorecard provides automated security-related checks for open-source repositories [19]. The NIST SSDF and SLSA explain why this security posture matters for dependencies [4], [5]. The checks describe present security practices, not future maintenance activity.

OSS Risk Radar therefore presents OpenSSF Scorecard as a parallel security-posture signal and excludes it from the inactivity-model feature vector (Section 4.1). No claim is made that these checks improve inactivity prediction; they provide complementary triage context.

3.4 Responsiveness Measurement Caveats

The responsiveness features cover issue first-response time and pull-request response and merge latency. Bot-generated responses can distort these measures [15], so they are treated as reconstructed GH Archive proxies, not canonical CHAOSS metrics (Section 2.3).

3.5 Research Gap

Prior work operationalizes inactivity and abandonment, standardizes health metrics, provides security-practice indicators, and predicts maintenance status [26], [27], [28]. The open question is whether the operationalization in Section 3.1 supports a reproducible, forward-looking, *calibrated* per-repository score within a dependency-triage artifact.

4 Methodology

The methodology combines a Design Science Research-oriented artifact with historical supervised learning. This approach fits the development and evaluation of a software artifact for a practical engineering problem [11], [12]. The artifact, OSS Risk Radar, supports open-source dependency maintenance and software supply-chain risk triage [29].

The training dataset uses real GitHub-hosted repositories and historical GitHub events, not manually assigned or synthetic labels (Section 1.3). For each repository, the builder creates observation snapshots whose features were available at or before date t . The label comes from the following 12 months, $(t, t + 12 \text{ months}]$. The resulting task estimates future inactivity from information available at the observation date.

4.1 Research Design

The research design has three parts: OSS Risk Radar ingests dependency or repository information and produces inactivity-risk scores from public signals; a historical dataset supplies supervised examples; and quantitative evaluation measures discrimination and calibration, complemented by formative manual case review.

The artifact estimates inactivity risk from observable maintenance and activity. It is neither a vulnerability scanner nor a definitive trust score: inactive repositories may have no vulnerability, while active ones may still have security weaknesses. The output prioritizes repositories for manual review.

OpenSSF Scorecard is a *separate* security-posture signal in the enrichment workflow [19]. A Scorecard-derived quantity does *not* enter the observation-time feature set in Section 4.7. Security-practice indicators appear as parallel analyst context, and the predictive target remains restricted to activity and maintenance signals.

For each repository, OpenSSF provides an aggregate in $[0, 10]$ and checks for branch protection, code review, signed releases, least-privilege workflow tokens, and known vulnerabilities [19]. The service adjusts strong and weak checks and returns a 0–100 `security_posture_score`. The separate field is marked not model-derived when Scorecard is missing; it is not imputed.¹

¹Claude Opus 5/1 — see the Declaration on the Use of Artificial Intelligence, p. V.

4.2 Iterative Artifact Development

The first iteration validated snapshot construction, training, artifact export, and manual scoring with a small pilot run that contributes no reported result. Manual review found implausible scores: one operationally active project ranked too high, while lower scores in other cases seemed to require more contextual assessment. This formative error analysis led to separate features for activity, releases, contributor dynamics and concentration, responsiveness, and backlog pressure. It followed a build–evaluate–diagnose–refine cycle.

The second iteration produced the dataset and model artifacts used for the reported evaluation, expanding the empirical basis to the repository-first foundation dataset of Section 4.4, whose seed buckets serve only as sampling provenance while the label is derived from future repository behavior.

4.3 Definitions and Prediction Target

The unit of analysis is a GitHub repository. Each row represents a repository at observation date t . The binary prediction task is:

Given the observable repository and package state at time t , predict whether the repository becomes inactive within the following 12 months.

The observation date lies far enough in the past for the following twelve months to have *already elapsed*. A snapshot can only receive a supervised label after this outcome window closes. Observation dates run quarterly from 2023-01-01 to 2025-04-01, and the GH Archive files extend to 2026-06-16. The final label window closes on 2026-04-01, within the recorded period (Section 4.6).

Each snapshot splits the historical archive at t into two disjoint periods:

- **Backwards from t** lies the evidence the model is allowed to see. Observation-time features aggregate the event history over the 30, 90, and 365 days before t (Section 4.7). This is the *input*.
- **Forwards from t** lies the twelve-month window from which the label is computed, and which the model never sees for that row. This is the *prediction target*.

Here, *future* is relative to t : the window follows the observation date but precedes dataset construction. Its outcome was unknown at t , although it is available for supervised training later. Live scoring sets t to the current date, for which the outcome window has not elapsed and no

label exists.²

The exported *inactivity label*, `label_inactive_12m`, is the negation of the future-window maintenance variable `maintained_12m`. The archive records positive evidence *of* maintenance: commits, contributors, releases, and merged pull requests. Defining these four observable checks first makes the rule easier to verify and vary (Section 6.4). Negation is exact, so the two directions are equivalent. Only inactivity, the *absence* of that evidence, is exported and modeled as the positive class because the target is the risk that maintenance stops.³ Rows with incomplete future windows remain unlabeled and are excluded from fitting.

The future label window is defined as:

$$(t, t + 12 \text{ months}]$$

Figure 4.1 shows this temporal structure: the observation-time features summarize repository history up to t (Section 4.7), and the label is derived only from the window after it.

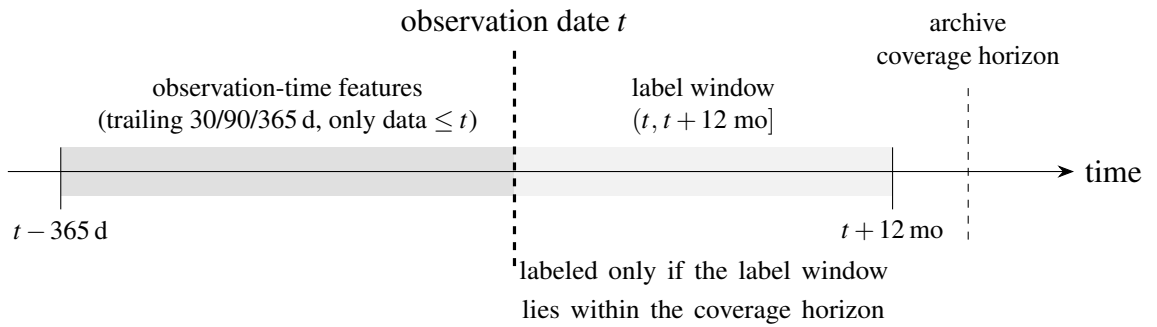


Figure 4.1: Temporal structure of a repository observation snapshot. The whole diagram lies in the past: the observation date t is chosen more than twelve months back, so that the label window after it has already elapsed and is recorded in the archive. Observation-time features are computed only from data at or before t , and the 12-month label only from the window after it; the dashed line at t is the temporal leakage boundary (Section 4.9). A snapshot is labeled only when the archive-wide coverage horizon spans its full label window (Section 4.6).

The current implementation treats a repository as maintained within the 12-month horizon if it is not archived or deleted by $t + 12$ months and if at least two of the following future-window activity checks are fulfilled:

1. human commits occur in at least three distinct months,
2. at least two contributors are active,
3. at least one release or package version publication occurs,

²Claude Opus 5/4 — see the Declaration on the Use of Artificial Intelligence, p. V.

³Claude Opus 5/4 — see the Declaration on the Use of Artificial Intelligence, p. V.

4. at least two pull requests are merged.

This is the *two-of-four maintenance rule*. Requiring two checks prevents a single drive-by commit or automated release from establishing maintenance. Requiring all four would misclassify healthy projects that do not release or receive external pull requests during a given year. The rule remains measurable from public history.

Merged pull requests proxy broader maintainer interaction. GH Archive records typed events, not resolved roles [30]; identifying maintainer comments would require attribution logic and remains vulnerable to bot distortion [15]. Merge events are unambiguous in the event stream. The artifact documents the implementation [31].

The offline build supplies no `archived_at` timestamp (Section 4.4), so the archival condition never fires and the label uses only the two-of-four rule. Archived repositories normally receive none of the four activity signals and are already marked inactive, making this a conservative simplification. The condition would, however, miss a repository archived while still receiving activity.

The two-of-four requirement and the cut-offs for months, contributors, releases, and merged pull requests are heuristic design choices, not externally validated constants. They require sustained activity. Because they affect class prevalence, Section 6.4 (Appendix A.13) repeats the evaluation with stricter and more permissive variants.

4.4 Data Sources

GH Archive is the primary historical source. It distributes public GitHub event streams as hourly archives [30], which the builder parses into normalized histories of commits, issues, pull requests, releases, stars, and forks.

deps.dev supplies package and dependency metadata when package-to-repository mapping is required [32]. GitHub supplies stable fields such as creation time, default branch, and fork state. OpenSSF Scorecard remains a separate security-practice source [19] (Section 4.1). Present-day GitHub activity totals are excluded from historical snapshots because they would leak information from after the observation date.

The first construction stage produces the *foundation seed*, a fixed list of 5,000 repository identities without features or labels. The second stage replays each history at ten observation dates. Storing the seed makes reruns use the same repositories instead of a new search result.⁴

The seed contains public, non-fork GitHub repositories with a license object and at least 100 stars by default. Separate active, dormant, and archived buckets control the mixture. They are

⁴Claude Opus 5/4 — see the Declaration on the Use of Artificial Intelligence, p. V.

neither features nor labels: labels come exclusively from activity in the 12-month future window [31]. They are, however, defined on repository state after the observation dates, which is qualified below. The target mixture is 45% active and 55% dormant or archived.

The dormant and low-star active buckets could not be filled, so a fallback added active repositories. The realized seed is 52.0% active, 28.0% dormant, and 20.0% archived (Appendix A.4), so the intended oversampling is only partly realized at seed level. The held-out inactive rate reported in Chapter 6 is therefore more relevant than the seed buckets. It exceeds reported unsampled rates: Coelho et al. observe that 16% of 2,927 active projects became unmaintained within one year, while Miller et al. report 15% abandonment among widely used npm packages over six years [2], [27]. The probabilities are calibrated to the sampled prevalence, not a deployment population (Sections 4.11 and 4.12).

The frame is sampled retrospectively. The seed was generated on 2026-06-11, after every label window had closed, and its bucket queries read repository state at that date. The active buckets require a push within the preceding year (`pushed: >=2025-06-11`), the dormant buckets require no push for two years (`pushed: <=2024-06-11`), and the archived buckets require the archive flag to be set. Entry into the frame is therefore decided by behavior that falls partly inside the outcome windows the model predicts. The buckets do not enter the feature vector and do not define the label, but they are not independent of it either. In the final test cohort, the inactive rate is 1.000 across the 1,399 dormant rows and 0.942 across the 1,000 archived rows. It is 0.409 across the 2,095 active rows and 0.209 across the 506 fallback rows. The dormant rate reaches 1.000 from the 2024-07-01 cohort onward.

Two consequences follow, and both restrict how the reported numbers may be read. The inactive rate of 0.661 describes this construction rather than a base rate obtainable by sampling repositories at t . The aggregate task is also easier than the deployment task. Much of it consists of separating buckets that were formed from later behavior and are already legible at t through commit recency. Consistently, the trivial recency baseline of Section 6.1.1 reproduces most aggregate ranking skill. All reported metrics are conditional on this retrospectively sampled frame. Chapter 8 returns to the threat, and drawing the frame from repository state at each observation date, rather than at generation time, is recorded as future work in Chapter 9.⁵

Offline repository metadata and its consequences. The reported build uses the stored seed and GH Archive files without live GitHub, registry, or `deps.dev` calls. It is repeatable without credentials, rate limits, or changes in mutable metadata. All observation-time features are reconstructed from event history, so the predictive content is unaffected. The offline path leaves creation date, archival timestamp, and fork flag unpopulated. This makes `repo_age_days` identically zero and the archival label condition and fork filter inert. Appendix A.5 details these effects; Chapter 9 lists metadata enrichment as future work.

⁵Claude Opus 5/11 — see the Declaration on the Use of Artificial Intelligence, p. V.

4.5 Dataset Construction Workflow

Figure 4.2 shows the construction sequence. The builder loads the foundation seed, resolves repository identities, enriches stable metadata, normalizes GH Archive events, and creates quarterly snapshots. For each snapshot, it computes features from data at or before t and the 12-month label from $(t, t + 12 \text{ months}]$. It exports both the training table and a feature cache for runtime full-history scoring.

Feature and label computation read disjoint parts of the normalized timeline and meet only in the exported row. Appendix A.10 lists the intermediate files that make each stage inspectable. The final table contains one snapshot per row, the features from Section 4.7, and one label column. It matches the existing training interface, so reconstructed data use the same training path as other runs. Appendix A.12 gives the rebuild commands, run documentation, and dataset hash.

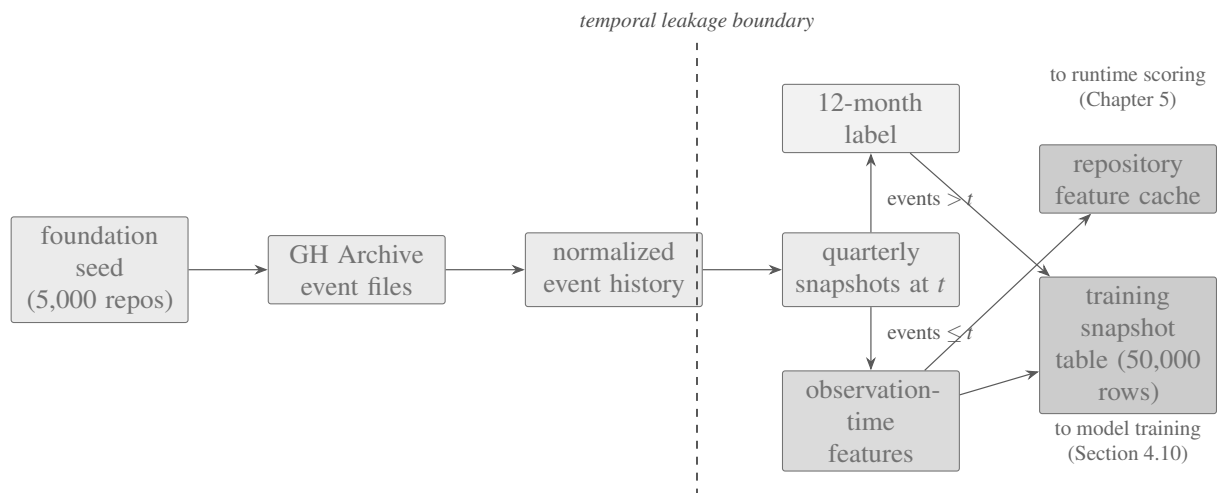


Figure 4.2: Dataset construction pipeline. The upper chain is ingestion: the fixed foundation seed selects the repositories, GH Archive supplies their raw events, and these are normalized into a per-repository chronological history from which quarterly observation snapshots are cut. At each snapshot the pipeline splits. Everything at or before the observation date t may become a *feature*; everything after it may only become the *label*; nothing crosses the dashed boundary (Section 4.9). The two paths meet only in the exported row. The feature path is written a second time, without labels, as the repository feature cache that lets the runtime score a previously seen repository in the full-history regime. Appendix A.10 lists the individual steps and the intermediate file written at each of them.

4.6 Observation Window and Historical Coverage

The reported run contains 50,000 rows: 5,000 repositories at ten quarterly dates from 2023-01-01 to 2025-04-01. Its GH Archive files span 2021-01 to 2026-06-16. The latest label window ends on 2026-04-01, so all 50,000 rows are labeled.

The time-ordered split assigns the oldest 37,500 rows to training, the next 7,500 to validation, and the newest 5,000 to testing. Because the ten cohorts have equal size, the 10% test partition is the final observation date; every test row is observed on 2025-04-01.

The cut is made by row position rather than by cohort, so the boundary between training and validation falls inside the 2024-10-01 cohort: 2,500 of its rows are assigned to training and 2,500 to validation. Training thus covers 2023-01-01 to 2024-10-01 and validation 2024-10-01 to 2025-01-01. Rows are ordered only by observation date and ties keep their original order, so the division within that cohort is by position and not by any repository property. The test partition is unaffected and remains one complete cohort.

Labels are used before they could have been observed. Each label requires the twelve months following its observation date. The latest training rows are observed on 2024-10-01 and their labels close on 2025-10-01; the latest validation labels close on 2026-01-01. Both lie after the test observation date of 2025-04-01. An analyst standing at 2025-04-01 could therefore not have fitted this model, because part of its training signal had not yet occurred. The evaluation is a retrospective chronological one with overlapping outcome windows rather than a simulation of prospective deployment. It supports the claim that the model ranks later observations correctly when fitted on earlier ones; it does not establish that the same model could have been built in time to issue those predictions. Removing the overlap would require an embargo of one full label horizon, which here means discarding the three cohorts whose outcome windows extend past the test date — 2024-07-01, 2024-10-01, and 2025-01-01, or 15,000 of the 45,000 non-test rows. With ten cohorts available this was judged too costly for the reported run and is recorded as future work (Chapter 9).⁶

The model is fitted on earlier observations and tested on later ones, matching the forecasting setting. However, each repository contributes ten snapshots and therefore appears in training and testing at different dates. Performance could partly reflect recognition of seen repositories. A repository-disjoint evaluation assigns every snapshot of a repository to one partition, leaving only unseen repositories in its test set. Appendix A.9.2 describes this split, and Section 6.3.2 reports the result.

A snapshot is labeled only if the archive covers the full twelve months *after* t , shown on the right of Figure 4.1. Other rows are retained but excluded from fitting. Because inactivity is an absence of events, incomplete coverage could otherwise turn missing data into positive labels.

This run contains no such rows: coverage ends on 2026-06-16, after the latest label window closes on 2026-04-01. An observation after roughly 2025-06, or missing archive files, would activate the gate. It removes no rows from the reported dataset.⁷ A smaller subset is used only for local pipeline smoke tests and never for reported results. Thus, all 50,000 reported rows pass

⁶Claude Opus 5/11 — see the Declaration on the Use of Artificial Intelligence, p. V.

⁷Claude Opus 5/4 — see the Declaration on the Use of Artificial Intelligence, p. V.

this coverage gate.

4.7 Feature Engineering

Feature set `feature-set-v4-full-history` contains 60 inputs derived before t . Of these, 43 reconstruct commit and contributor activity, issue and pull-request flow, releases, risk proxies, responsiveness, and backlog pressure. The other 17 describe the current snapshot, including dependency context, popularity, and ecosystem. The cold-start regime uses these 17 features (Section 4.8).

Backlog pressure comprises three historical quantities: relative open-issue growth over the preceding 90 days, issues older than 90 days that remain open at t , and their share of the open backlog. The cold-start set also contains backlog size and 90-day open-issue growth. Together they indicate whether open items are growing and ageing independently of response speed.

Because the seed is repository-first and the offline build makes no registry call, ecosystem indicators and dependency-context flags are constant. Eight of the 60 inputs are constant across all 50,000 rows and are inert, because zero-variance columns cannot split a tree. Two further inputs, `package-age-days` and `versions-published-365d`, vary only with the observation date and not between repositories, so ten inputs in total carry no repository-level signal and roughly 50 remain informative. The two counts describe the implemented vectors, not the number of informative signals. Appendix A.5 lists the affected features and explains why the two cohort-dependent inputs cannot change the reported ROC-AUC.⁸ Appendix A.2 (Table A.1) maps the health-indicator categories of Section 2.2 to feature groups. Appendix A.3 (Tables A.2, A.3, and A.4) defines every feature.

The builder parses GH Archive events into chronological histories of commits, issues, pull requests, releases, stars, and forks [30]. Features are counts over the 30, 90, and 365 days before t , state at t , or derived ratios, latencies, and drop-off measures. They lie before the leakage boundary in Figure 4.1. Trailing windows capture trajectory instead of lifetime volume.

Commit and contributor counts apply a *human-actor filter* that excludes events attributed to bot accounts. It prevents automated activity, such as weekly dependency-bot commits, from appearing as maintenance. The label uses the same filter, and Appendix A.16 measures its effect by rebuilding the dataset without it. Relative and trend-based features supplement absolute counts that do not compare across project sizes.

Revision activity predicts project survival [14], which motivates the commit, contributor, and release windows. Issue, pull-request, and responsiveness signals follow the CHAOSS project-health vocabulary [17], [18]. Since individual context-free indicators are insufficient across

⁸Claude Opus 5/6 — see the Declaration on the Use of Artificial Intelligence, p. V.

heterogeneous projects [16], the model learns a calibrated combination.

The responsiveness features are *reconstructed proxies*, not the canonical CHAOSS *Time to First Response* metric. They follow thesis-specific rules over GH Archive events. A first response is the earliest comment from an account other than the person who opened the issue or pull request. This account need not be a maintainer and may be a bot [15]. For pull requests without comments, the merge or close event substitutes for a response.

Reconstructed star and fork counts use a second sense of *proxy*: they are lower bounds when archive coverage begins after repository creation. These limitations do not invalidate the features, but attributions describe observable behavior rather than canonical CHAOSS measurements.⁹

Reconstructed issue backlogs likewise describe the archived tracker, not a full API result.

Undefined versus zero-valued features. Zero is the *most favorable* value for several variables and must not encode missing data. When no commit is observed, “days since last commit” is *right-censored* at the age of the observable record. This represents maximal observed staleness instead of a recent commit. A metric that does not exist, such as median merge time for a repository with no merged pull request, remains *undefined* for model-specific imputation. Appendix A.5 defines the censoring bound, imputation, and two “no events” indicators.

This distinction keeps data availability from masquerading as maintenance behavior in the attributions of Section 6.3.1. A zero-valued latency would make silence look like immediate responsiveness. The revised feature set also excludes unreliable point-in-time predictors (Section 4.2). For example, `repo_archived_at_obs` remains diagnostic metadata while already-archived rows are excluded from fitting, preventing archival from becoming a shortcut label.

4.8 Full-History and Cold-Start Prediction Regimes

Each model is trained in two regimes. *full-history* uses all 60 inputs, including 43 reconstructed commit, release, contributor, issue, pull-request, and activity-drop features before t . *cold-start* uses the 17 current-snapshot features and requires no event history.

This ablation on *feature availability* measures the value of reconstructing history from tens of thousands of public archive files compared with one metadata read. Historical features are theoretically available for any public repository: the regimes do not contrast what a scorer *can* know with what it *cannot*. However, 7,687 of 50,000 rows have no earlier observed event, leaving their historical features censored or undefined (Section 4.7). At runtime, repositories absent from the feature cache also use cold-start, although this is not the motivation for the ablation. The comparison asks what reconstruction is *worth*.

⁹Claude Opus 5/4 — see the Declaration on the Use of Artificial Intelligence, p. V.

Across the full test set, full-history exceeds cold-start by only 0.005 ROC-AUC. Many repositories are already quiet at t , so recency makes their outcome easy to rank without reconstructed history. A repository with no commits for two years, for example, needs no reconstructed trajectory to appear at risk. Section 6.1.2 shows that this depends on *which repositories the test set contains*: the difference is larger among repositories that remain active at t and later decline.

4.9 Leakage Prevention

Figure 4.1 shows the boundary at t between feature construction and label derivation. The builder enforces:

1. Observation-time features are derived only from timestamps $\leq t$.
2. Labels are derived only from timestamps in $(t, t + 12 \text{ months}]$.
3. Rows with incomplete future-window coverage remain unlabeled.
4. Live GitHub enrichment is limited to stable repository metadata such as creation time, default branch, and fork state.
5. Historical stars, forks, issues, pull requests, and releases are not taken from current GitHub API totals.
6. Seed buckets such as active, dormant, and archived are used only for sampling provenance and not as labels. They are drawn from repository state at seed-generation time and therefore constrain the frame rather than the row (Section 4.4).

At t , later activity is unknown. It is excluded from features and used only to derive the label.

Activity persistence is distinct from leakage. Days since the last commit, activity drop, and days since the last release use valid information available at t , but already-quiet repositories tend to remain quiet. Some discrimination therefore reflects persistence instead of early decline. This motivates the recency baseline and active-at- t evaluation in Section 4.11.

4.10 Model Training

One notebook performs all fitting and exports the model files. The scoring service only loads these files [29]. Section 5.4 describes this reproducibility boundary.

The learned families are Logistic Regression, XGBoost, and a feedforward neural network, each trained in both regimes. Logistic Regression provides an interpretable linear model. XGBoost supplies non-linear gradient-boosted trees [33]; the neural network learns non-linear interactions

through hidden layers [21].

Two non-learned baselines rank repositories by days since the last human commit or by negated annual commit volume. They select thresholds on validation data and are evaluated on the test set. Commit volume covers all test rows, while recency is defined for 4,556 rows (Section 6.1.1). Both run inside the training notebook, linking their results to the same run as the learned models. Because the label is activity-related, these baselines measure how much ranking skill one activity signal already provides (Section 4.9). Section 6.1.1 shows that the learned advantage is concentrated among repositories still active at t .

The exact model configurations and hyperparameters are documented in Appendix A.7.4; all learned models are calibrated on the validation set before evaluation.

Training uses a time-aware split with the default ratio:

75% training, 15% validation, 10% test

Rows are ordered by observation date. The earliest rows train the model; validation predictions fit the histogram calibrator and select the decision threshold. The final test set is reserved for reporting. This avoids mixing later observations into earlier model development.

4.11 Evaluation Strategy

The primary evaluation uses the time-aware test set for neither fitting nor calibration. Appendix A.11 defines accuracy, precision, recall, F1, Brier score, log loss, ROC-AUC, average precision, expected calibration error, and the combined quality score.

The inactive base rate is approximately 0.66, so ROC-AUC is a reasonable primary ranking metric. Precision-recall summaries are more informative under strong class imbalance [24], which is why average precision is reported for the active-at- t subset defined below, where the inactive rate falls to 0.225 (Section 6.1.3). Precision and recall are reported at the selected operating point. Since precision depends on prevalence, it is conditional on this sample and is not a deployment guarantee or headline comparison. Calibration is assessed separately because probability scores must match observed frequencies [25].

The models are compared with recency and evaluated separately on repositories still active at t . This subset limits domination by activity persistence (Section 4.9) and represents the harder case in which early warning can still affect a decision. Active-at- t means at least one bot-filtered human commit in the trailing 90 days: $\text{active-at-}t = \mathbf{1}(\text{commits}_{90d} > 0)$. The 90-day window is an operational choice, not a tuned threshold. A multi-signal variant with recent releases or merged pull requests remains future work.

Because inactivity as defined in Section 4.3 is an operational proxy, formative manual review identifies context-sensitive cases and predictions made for the wrong reason; it drove the first feature revision. A systematic case review is future work (Chapter 9). The held-out split is an *internal* evaluation of temporal generalization. Chapter 8 discusses external validity.

The staged full-history artifact is `xgboost-full-history`, version 0.2.0, trained with `feature-set-v4-full-history`. Chapter 6 compares its held-out metrics with Logistic Regression, `neural-net-full-history`, and the two baselines.

4.12 Methodological Limitations

This section records data and design limitations; Chapter 8 discusses the broader validity threats.

Missing GH Archive hours leave histories incomplete. Public GitHub events omit development elsewhere, off-platform releases, and stable software requiring little visible activity. Package-to-repository mappings can be ambiguous after moves or with incomplete metadata. The inactivity label is an operational proxy, and its heuristic thresholds affect prevalence (Section 4.3).

The seed covers visible, licensed repositories above a minimum star count, not all of GitHub. Oversampling dormant and archived repositories makes probabilities conditional on the sampled prevalence; deployment risk estimates require recalibration (Section 4.4).

The internal split tests temporal generalization within the dataset. External validation on a newly sampled repository set remains future work (Chapter 9).

5 Implementation

This chapter describes the system architecture, historical dataset builder, design-cycle corrections, and evaluation instrumentation. Appendices A.6 to A.9 document technology choices, module layout, model configurations, artifact format, runtime services, and deployment. This separation keeps the implementation reproducible and auditable under the Design Science Research framing [11], [12].

5.1 System Architecture Overview

Figure 5.1 shows the organization. Three properties affect the research results; Appendix A.6 gives the component inventory and language choices.

1. **One shared definition of every feature.** The training code and the runtime scoring service read the same versioned feature definitions. A feature computed one way during training and another way at scoring time would silently invalidate every reported metric, because the model would be evaluated on one function of the data and deployed on another.
2. **Training and scoring are strictly separated.** Models are never fitted during normal application use: the training pipeline exports versioned model files that are promoted into a staged bundle, and the scoring service loads only these. This is what allows Chapter 7 to be read as evidence about the evaluated model.
3. **The unit of analysis is a single repository,** scored independently; the dependency inventory to grade is an external input, for the reasons set out in Section 2.1.

The source code is public, and all paths are relative to the repository root. Appendix A.12 pins the evaluated revision and gives the rebuild and training commands [29].

5.2 Historical Dataset Builder

The historical builder is a Python package under `app/training/maintenance_dataset`. It converts a repository seed and GH Archive files into an inspectable, leakage-controlled dataset. Separate modules handle ingestion, event parsing, features, and labels for independent testing. Appendix A.7.1 documents the layout, and Appendix A.7.2 the quality report.

The code enforces the observation-time/future-window boundary. Each snapshot has a feature window for the year before t , a previous window for the preceding year, and a label window for

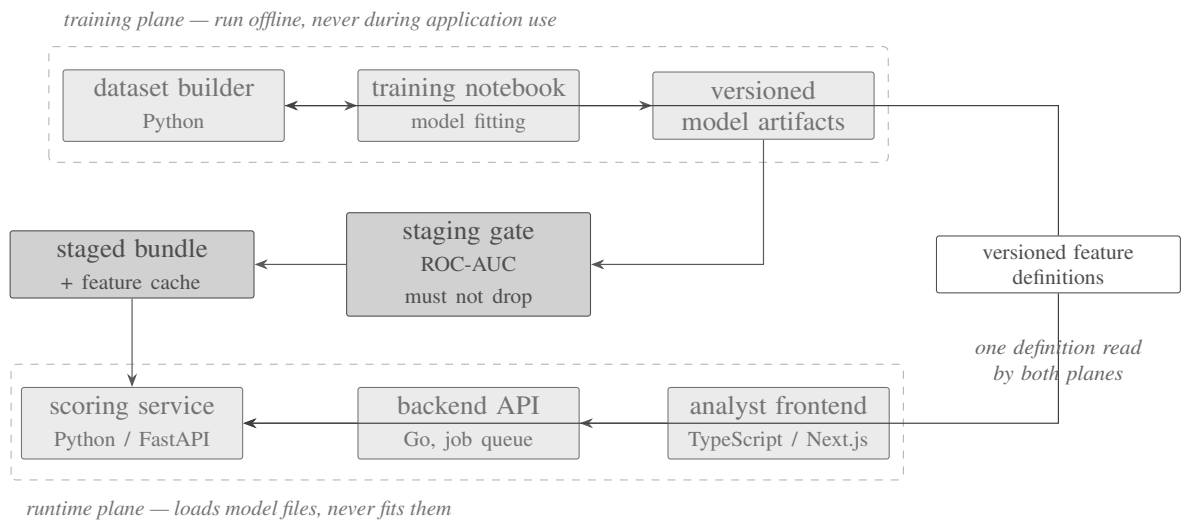


Figure 5.1: System architecture. The artifact separates into two planes that never overlap in responsibility. In the *training plane* the dataset builder produces the historical table, the notebook fits the models, and the run is serialized into versioned artifacts. Those artifacts reach the *runtime plane* only through the staging gate, which promotes a candidate bundle into the staged bundle only if it does not regress against the currently promoted baseline (Section 5.4); the runtime loads these files and never fits a model. The element crossing both planes is the versioned feature definition set: the builder and the scoring service compute every feature from the same definitions, which is what makes a metric measured in the training plane meaningful for a score produced in the runtime one. The component inventory and language choices are given in Appendix A.6 (Table A.6).

the twelve months after t . Features use events at or before t ; labels use events strictly after it.

The label implements the thesis-specific two-of-four rule from Section 4.3. It counts distinct human-commit months, future contributors, releases or package publications, and merged pull requests. A repository is maintained only when it is not archived or deleted within the horizon and at least two conditions hold. If the dataset-wide archive horizon does not cover the full label window, `future_window_incomplete` is set and the row remains unlabeled. Section 5.2 describes a correction to this coverage horizon.

Human-versus-bot attribution is applied during both feature and label computation. Commit and contributor counts use an actor filter that excludes automated accounts, so that bot-driven activity does not inflate maintenance evidence. The same reasoning motivates the cautious treatment of responsiveness signals in the methodology, where automated responses are known to distort interaction metrics [15]; those features are reconstructed proxies and are not bot-filtered (Section 4.7).

The export step produces two outputs with different purposes. The training snapshot file is the dataset itself — one row per observation snapshot, in the table layout the training notebook already expects (Section 4.5). The repository feature cache stores, for each repository, the most recent observation-time feature row, with diagnostic-only fields such as `repo_archived_at_obs` removed. This cache is what later allows the runtime backend to score a previously seen repository in the richer full-history regime rather than the cold-start regime.

The formative review produced two builder refinements. Raw counts such as absolute open issues, which can also be high for healthy projects, were replaced with relational and trend-based variables. The builder also resolves one primary package-to-repository link so mirrors do not make an active project appear inactive (Appendix A.7.3). The second refinement corrected label completeness.

The label-completeness correction. This correction changes only which rows qualify as labeled examples. The prediction objective, window $(t, t + 12 \text{ months}]$, two-of-four rule, model, and target are unchanged.

A row is labeled only when its future window is fully covered (Section 4.9). The initial implementation used each repository’s last event as its coverage end. This proxy is correlated with the outcome: a repository that falls quiet has no late events, so later snapshots were discarded as “incomplete” even when the archive covered their windows. Inactive repositories were therefore under-represented among labeled positives.

The corrected gate uses a dataset-wide archive horizon: either the latest event across all repositories or the cutoff supplied by `--coverage-end`. The orchestration script defaults to the latest day of complete hourly GH Archive coverage. Silent repositories inside the horizon can now be labeled inactive; rows extending beyond available data remain unlabeled. The `repo_`

`silent_before_horizon` diagnostic identifies labelable rows whose own last event precedes the horizon, making the rescued share inspectable in the quality summary. This aligns the implementation with the leakage rule and removes the selection effect that reduced the labeled inactive population.

5.3 Model Training and Evaluation Instrumentation

Both regimes implement Logistic Regression and a feedforward neural network from first principles, and XGBoost through its library [33]. The first two remain dependency-light and inspectable. All families share the dataset, time-aware split, calibration, and evaluation code. Appendix A.7.4 gives the configurations, and Appendix A.7.5 the artifact format.

All families use the same post-processing. A classifier may rank repositories correctly while over- or understating inactivity probability. A separate calibration step maps raw scores to observed frequencies (Section 2.4).

The *histogram* calibrator groups validation predictions into probability bins, assigns each a smoothed empirical inactivity rate, and enforces a monotonic mapping. Unlike logistic (Platt) scaling, it assumes no parametric shape. This suits model families with different distortions. Its bin boundaries and rates are also stored in the model file and replayed at inference. The calibrator uses validation data, never test data, so calibration metrics remain out-of-sample.¹

Inference uses piecewise-linear interpolation between populated bins. A flat lookup would collapse every raw score in a bin to one value and create ties in the ranked view. Interpolation preserves raw-score order and empirical calibration. Monotonic bin rates keep the mapping monotonic.

A decision threshold is then selected on the calibrated validation predictions by maximizing the F1 score, and the evaluation module computes the reported metrics (Section 4.11) on the test set.

The first complete run motivated two additional evaluation instruments.

Most test rows describe repositories already quiet before t . The *slice evaluation* checks whether this easy majority dominates the aggregate result. It recomputes metrics by popularity, history regime, and activity at t , so a strong majority cannot hide a weak subgroup. Appendix A.9.1 documents the included and excluded dimensions. Section 6.1.1 reports the resulting split finding.

The *repository-disjoint split* addresses recurrence across the date-based split. It confines all ten quarterly snapshots of a repository to one partition, testing generalization to unseen repositories (Section 6.3.2).

¹Claude Opus 5/4 — see the Declaration on the Use of Artificial Intelligence, p. V.

The two instruments differ in kind. The slice evaluation is a re-analysis: it re-scores the predictions of the already fitted model. The repository-disjoint split is a separate training run, which refits preprocessing, model, calibrator, and decision threshold on its own partitions and leaves its test rows untouched until every fitting decision is made. Neither changes the staged artifacts or the headline metrics, because the reported run keeps the date-based split. Appendix A.9 documents their mechanics.²

5.4 Reproducibility of the Evaluated Artifact

Three mechanisms make each result traceable to a fixed model and dataset.

All training runs through `notebooks/oss-maintenance-training.ipynb`, the only writer of model files. `npm run ml:train` executes it non-interactively (Appendix A.7.6), so reported results use the documented path. Each artifact records ordered feature names, parameters, calibration bins, decision threshold, feature-set version, and dataset hash (Appendix A.7.5). The hash in Appendix A.12 verifies rebuilt data against the evaluated dataset.

Artifacts are not deployed automatically. The staging command requires every model's ROC-AUC not to drop and its Brier score not to exceed the configured tolerance relative to the deployed baseline (Appendix A.7.7). Dataset hash and model version link the demonstrator output in Chapter 7 to the evaluated model in Chapter 6.

The scoring service (Appendix A.8.1), backend (Appendix A.8.2), frontend (Appendix A.8.3), and deployment (Appendix A.8.4) consume these artifacts. They provide engineering context; no reported result depends on them.

²Claude Opus 5/4 — see the Declaration on the Use of Artificial Intelligence, p. V.

6 Results and Discussion

This chapter evaluates predictive performance, calibration, feature attribution, errors, and robustness to repository recurrence and the label definition. Aggregate performance is compared with the active-at- t subset to separate activity persistence from early-warning skill.

6.1 Predictive Performance

Table 6.1 gives held-out metrics for three learned families in both regimes and two non-learned baselines. All learned models use the same time-aware split (75/15/10; $n_{\text{test}} = 5,000$, inactive base rate 0.661) and validation-set histogram calibration. Differences therefore reflect model family. The combined quality score summarizes ROC-AUC and Brier for comparison; it is not standalone evidence. Appendix A.11 defines the metrics.

Table 6.1: Held-out test metrics by model family and feature regime, with the two trivial baselines. Higher is better for ROC-AUC, F1, and quality; lower is better for Brier and ECE. The best value within each regime is shown in bold. *The recency baseline is defined on the 4,556 test rows that have an observable commit history; see the text for the like-for-like comparison.

Model	ROC-AUC	Brier	ECE	F1	Quality
logistic-regression-full-history	0.949	0.088	0.047	0.911	0.782
xgboost-full-history	0.958	0.078	0.035	0.916	0.810
neural-net-full-history	0.956	0.081	0.028	0.913	0.803
logistic-regression-cold-start	0.932	0.100	0.044	0.897	0.740
xgboost-cold-start	0.953	0.082	0.033	0.909	0.798
neural-net-cold-start	0.951	0.084	0.027	0.908	0.791
recency-baseline (days since commit)	0.957*	—	—	0.918	—
commit-volume-baseline (—commits_365d)	0.897	—	—	0.904	—

The ordering is stable across regimes. The full-history XGBoost model has ROC-AUC 0.958 and Brier 0.078, followed by the neural network (0.956 / 0.081) and Logistic Regression (0.949 / 0.088). The neural network has the lowest calibration error but does not improve ranking or Brier over XGBoost. It also costs more to train and is less interpretable, consistent with evidence favoring trees on moderate tabular data [34]. Logistic Regression is retained for coefficient attribution, XGBoost for runtime scoring, and the neural network as a tested alternative.

Cold-start is only marginally weaker for the non-linear models. `xgboost-cold-start` reaches 0.953 against 0.958 with the smaller feature set. Logistic Regression falls from 0.949 to 0.932. The cold-start vector retains `last_push_age_days`, a strong substitute for reconstructed commit

history on this mixed test set, although it lacks the commit-window features that dominate full-history gain (Section 6.3.1). The XGBoost gap is 0.005 in aggregate but 0.030 after removing the already-quiet majority (Section 6.1.2).

6.1.1 Comparison Against Non-Learned Baselines

Since the label is activity-derived, single-signal rules show how much ranking skill is available without learning (Section 4.10). The baselines use days since the last human commit and negated annual commit volume.

On the aggregate rows, recency reaches ROC-AUC 0.957 and F1 0.918; XGBoost reaches 0.958 and 0.916. This is not a like-for-like comparison. `days_since_last_commit` is undefined for 444 rows without prior observed events (Section 4.7), so recency covers 4,556 rows and XGBoost all 5,000.

Re-scored on the 4,556 shared rows, XGBoost reaches 0.974 against 0.957, a margin of 0.018. Increments use unrounded metrics before rounding and may differ from subtractions of displayed values. F1 was *not* recomputed on the shared rows and is not compared; no claim rests on the F1 column of Table 6.1. Commit volume is weaker on the full test set (0.897) but is the stronger trivial rule on recency-defined rows (Section 6.1.2).

At the inactive base rate of 0.661, recency already supplies most aggregate ranking skill. The sixty-feature model adds a modest increment *on a test set where most repositories are already quiet*. On the 1,794 repositories still active at t , XGBoost reaches 0.905 against 0.844 for the best trivial rule. This margin is 0.061, compared with 0.016 on the recency-defined rows (Section 4.11).

6.1.2 Early Warning: Removing the Already-Inactive Repositories

The active-at- t subset removes repositories already silent at the observation date. Its inactive rate is 0.225, compared with 0.661 for the full test set.

Comparing the two XGBoost regimes isolates the 43 reconstructed historical features (Section 4.8). Table 6.2 scores every arm on identical recency-defined and active-at- t rows. Rows without recency are dropped from *all* arms. The trivial column selects commit volume, the stronger rule in both sets: 0.958 against 0.957 for recency-defined rows and 0.844 against 0.792 for active-at- t . Section 6.1.1 instead uses recency specifically, so its margin has a different comparator.

On recency-defined rows, history adds 0.005 ROC-AUC and the 17 snapshot features add 0.011 over the best single signal. On active-at- t , the gains are 0.030 and 0.031; the historical gain is six

Table 6.2: Held-out ROC-AUC by feature-set size, on identical row sets.

Subset	n	Full-history (60)	Cold-start (17)	Best trivial rule (1)
Recency-defined	4,556	0.974	0.970	0.958
active@ t	1,794	0.905	0.875	0.844

times its aggregate value. The ordering from one to 17 to 60 features is nearly flat in aggregate and strictly monotone in the early-warning subset.

The learned model exceeds the best trivial rule by 0.016 on recency-defined rows and 0.061 on active-at- t . The full test set is dominated by repositories already quiet at t , so aggregate ROC-AUC largely measures activity persistence. Among repositories still active at t , XGBoost reaches 0.905 against 0.844; the repository-clustered bootstrap also supports this margin (Section 8.4). The learned contribution is therefore concentrated *among repositories that are active at the observation time*, while the aggregate advantage over a trivial rule remains small.¹

6.1.3 Precision-Recall Evaluation of the Early-Warning Subset

ROC-AUC measures pairwise ranking across both classes and is insensitive to class prevalence. Average precision summarizes the precision-recall curve for the positive class and uses the positive rate as its no-skill reference (Appendix A.11). It provides an additional view for active-at- t , whose inactive rate is 0.225 [24]. Table 6.3 reports it for the same held-out predictions and row sets as Table 6.2, with percentile intervals from 2,000 bootstrap resamples.

Table 6.3: Held-out average precision on identical row sets, with 95% bootstrap intervals. The no-skill column is the positive rate, which a random ranking attains.

Subset	n	No skill	XGBoost	Commit volume	Recency
Recency-defined	4,556	0.669	0.986	0.977	0.979
95% interval			[0.983, 0.989]	[0.973, 0.981]	[0.976, 0.982]
active@ t	1,794	0.225	0.726	0.599	0.512
95% interval			[0.679, 0.771]	[0.548, 0.649]	[0.464, 0.565]

The split finding of Section 6.1.1 is sharper under this metric than under ROC-AUC. On recency-defined rows, all three average-precision scores are high, and the model’s margin over the better trivial rule — recency under this metric, not commit volume as in Table 6.2 — is 0.007 [0.005, 0.009]. On active-at- t , the margin is 0.127 [0.096, 0.160] against commit volume, the better rule there, and 0.213 [0.159, 0.264] against recency. All intervals exclude zero. Relative to the no-skill line of 0.225, the model reaches 3.2 times the random rate and the better trivial rule 2.7 times.

¹Claude Opus 5/4 — see the Declaration on the Use of Artificial Intelligence, p. V.

The corresponding ROC-AUC margin is 0.061 (Table 6.2). Average precision supports the same conclusion and shows a larger separation between the model and the trivial rules. Because the principal comparison is summarized by the areas and bootstrap intervals, no precision-recall curve is included. The resulting loss of operating-point detail is discussed in Section 8.4. The analysis is reproduced by `scripts/ml/precision_recall_eval.py`, which reruns the reported configuration and aborts if the re-scored predictions do not reproduce that run’s held-out ROC-AUC (Appendix A.1). The rerun reproduces the staged run’s aggregate ROC-AUC of 0.957852, so Tables 6.2 and 6.3 describe the same fitted model.²

6.2 Calibration and Threshold Selection

Calibration is evaluated because the score is presented as a probability. The histogram calibrator uses validation data, never test data. For `xgboost-full-history`, Brier is 0.078, log loss 0.246, and ECE 0.035. At the selected threshold of 0.51, precision is 0.90 and recall 0.93. The neural network has the lowest full-history ECE (0.028); Logistic Regression has the highest (0.047). Calibration and discrimination therefore order the models differently.

Empirical inactivity increases monotonically across the ten calibration bins, from 0.032 in the lowest band to 0.988 in the highest. The two extreme bands contain 65.9% of validation predictions, leaving less support for mid-range probabilities. The demonstrator’s per-prediction evidence measure reflects this bin support (Chapter 7). Calibration is also conditional on sampled prevalence; live population probabilities require recalibration (Section 4.12).

6.3 Feature Importance and Error Analysis

This section compares standardized Logistic Regression coefficients with XGBoost gain, then examines repository recurrence, subgroup performance, and errors.

6.3.1 Feature Attribution

Logistic Regression provides transparent attribution for `xgboost-full-history`’s comparison model (Section 6.1). Standardized inputs make coefficient magnitudes comparable, and signs indicate the *direction* of association with inactivity. Figure 6.1 shows the leading coefficients.³

Sustained development is the strongest inactivity-lowering signal, followed by recent maintainer integration and any issue activity. The two staleness measures are the strongest inactivity-raising

²Claude Opus 5/12 — see the Declaration on the Use of Artificial Intelligence, p. V.

³Claude Opus 5/5 — see the Declaration on the Use of Artificial Intelligence, p. V.

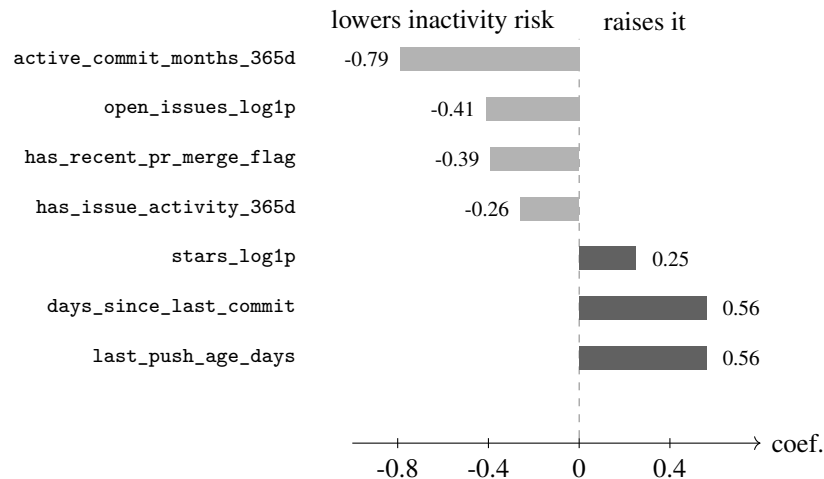


Figure 6.1: Standardized logistic-regression coefficients for the seven strongest *interpretable* features. Because the inputs are standardized the magnitudes are comparable; a negative coefficient means the feature is associated with a *lower* probability of inactivity. The two shaded groups behave as intended — sustained development lowers risk, staleness raises it — while `open_issues_log1p` and `stars_log1p` carry counter-intuitive signs discussed below. `package_age_days` is omitted although its coefficient (-0.26) would place it seventh by magnitude. Under the offline build, it takes one value per quarterly observation cohort (Appendix A.5). It therefore encodes the observation date rather than any property of the repository and is not interpretable as a maintenance signal.

signals (Appendix A.3).

Two coefficients have counter-intuitive signs. Larger issue backlogs associate with *lower* inactivity, while more stars associate with *higher* inactivity. These associations may be confounded: a backlog implies users who file issues, and stars may proxy unobserved project characteristics because repository age is unavailable (Section 4.4). The coefficients describe association, not causation; XGBoost provides ranking performance.

Before the encoding correction in Section 4.7, zero response latency marked projects with no issues or pull requests. Responsiveness medians then appeared among the strongest signals with a misleading meaning. After correction, they leave the leading coefficients, and `has_issue_activity_365d` and `has_pr_activity_365d` carry the absence of activity explicitly.

XGBoost gain is normalized over the 42 of 60 features used in at least one split. `commits_365d` (0.42) and `commits_90d` (0.20) provide over three-fifths of total gain. `active_commit_months_365d` (0.08) and `has_recent_pr_merge_flag` (0.07) follow; `last_push_age_days`, `opened_prs_90d`, `days_since_last_commit`, and `merged_prs_90d` form the next tier.

Both models emphasize recent sustained development. The linear model emphasizes active commit months, while XGBoost concentrates gain on commit-volume windows. Cold-start omits these windows, but `last_push_age_days` substitutes for them on the already-quiet majority.

The substitution is weaker on active-at- t (Section 6.1.2).

6.3.2 Robustness to Repository Recurrence

With every repository confined to one partition, the `xgboost-full-history` model is retrained, recalibrated, and rethresholded on that partitioning (Appendix A.9.2). ROC-AUC falls from 0.958 to 0.950 and Brier rises from 0.078 to 0.079. The small change bounds optimism from repository recurrence and indicates discrimination on repositories held out from the same foundation frame, not transfer to open-source repositories in general (Section 4.4). This check is necessary because ten equal quarterly cohorts make the chronological test set the final date, while every test repository also appears earlier in training. Section 8.2 treats this as an internal-validity threat.

6.3.3 Slice Evaluation

The `xgboost-full-history` predictions are re-scored by subgroup to identify variation hidden by the aggregate. Appendix A.15 (Table A.11) gives the full results.

On active-at- t ($n = 1,794$, inactive rate 0.225), XGBoost reaches 0.905 ROC-AUC with a 95% repository-clustered bootstrap CI of $[0.889, 0.921]$. Each snapshot in this test subset belongs to a distinct repository, so the interval equals an ordinary row bootstrap. Recency reaches 0.792 and commit volume 0.844. The 0.061 margin over the best rule has a 95% CI of $[0.046, 0.078]$, excluding zero, compared with 0.016 on recency-defined rows (Section 6.1.1). At the selected operating point, recall is 0.49 and precision 0.74. The model therefore ranks early decline but flags fewer than half of the repositories that later fall quiet.

Low-popularity repositories reach 0.846, compared with 0.972 and 0.964 for high and medium popularity (Table A.11). They require more caution, although “low” is relative to a seed requiring at least 100 stars. Per-prediction evidence support should surface this weakness. Repositories with reconstructable history reach 0.921, compared with 0.929 for cold-start-like rows. The latter subgroup has a much higher inactive rate (0.924 against 0.350), so this does not show that history is unhelpful. It only confirms that aggregate regime performance is close (Section 6.1).

6.3.4 Error Analysis

Two weaknesses remain. Low-visibility repositories provide sparse public signals. Evidence support flags this condition, but its relationship with prediction error has not been validated separately. On active-at- t , the F1-selected threshold also detects only about half of later inactivity. The score should therefore rank manual review, not gate decisions; it provides *no* automatic

acceptance rule.

6.4 Robustness to the Label Definition

The evaluation repeats the heuristic two-of-four rule with permissive one-of-four and strict three-of-four variants (Section 4.3). The harness re-labels all 50,000 snapshots, retrains full-history XGBoost, and exactly reproduces the primary run. The inactive rate ranges from 0.581 to 0.746, while full-test ROC-AUC remains between 0.958 and 0.961.

On identical recency-defined rows, the margin over the stronger trivial rule is 0.010, 0.016, and 0.023 as the rule tightens (Section 6.1.1; Appendix A.13). The learned margin therefore widens with the stricter rule, and the threshold choice does not produce the central ranking result. A systematic sweep over each cut-off remains future work.⁴

The human-actor filter applies to commit and contributor features and to the label. Disabling it changes ROC-AUC by only 0.0006–0.0011 across model families (Section 4.7). It changes the label in one direction: 357 of 50,000 snapshots switch from inactive to active, with none switching back. Bots can make a dormant project appear maintained but cannot make an active one appear inactive. The filter is therefore justified by construct validity, not predictive accuracy (Appendix A.16).⁵

⁴Claude Opus 5/9 — see the Declaration on the Use of Artificial Intelligence, p. V.

⁵Claude Opus 5/4 — see the Declaration on the Use of Artificial Intelligence, p. V.

7 Practical Demonstrator

This chapter applies OSS Risk Radar to repositories outside the foundation seed. It serves the Design Science Research demonstration phase and provides an out-of-distribution error analysis [11], [12]. The application can expose failure modes not visible in a test set drawn from the seed distribution.

The input is a real project’s dependency set. Scores come from the promoted artifact bundle (Section 5.4). For a shared dataset hash and regime, the backend scores each staged model evaluated in Chapter 6 and averages the outputs into an *ensemble profile* (Appendix A.8.2). Every reported score is the mean of two models; their spread becomes the model-agreement factor.

Chapter 6 evaluates individual artifacts, not this ensemble. Comparison with its strongest member, `xgboost-full-history`, remains future work (Chapter 9). Furthermore, 29 of 30 repositories use the weaker cold-start regime. This coverage limitation motivates the cache-expansion work in Chapter 9. The chapter therefore analyzes the deployed configuration rather than measuring predictive accuracy again. Outputs reflect public repository state on 2026-07-09.

7.1 Example Repository Set

Each repository is scored *individually*, and software-composition-analysis tooling supplies the external inventory [35] (Section 5.1).

The example is ArchipelagoMW/Archipelago, a large, active open-source multi-world game randomizer. Its declared Python dependencies map to 30 source repositories, from `flask`, `jinja2`, and `werkzeug` to smaller packages. Dependency resolution and graph construction are outside the artifact. Each declared repository is scored independently, and the demonstrator orders the 30 scores without modeling relationships between dependencies.¹ This mixture was chosen to avoid a uniformly low-risk example.

The workflow enriches live metadata and applies the staged ensemble. Only `flask` is present in the feature cache and receives full-history scoring; the other 29 repositories use cold-start (Section 4.8). This shows the coverage limit for arbitrary submissions. Chapter 6 shows that on active-at-*t* repositories, full-history exceeds cold-start by 0.030 ROC-AUC (Section 6.1.2), and most example repositories are active. Cold-start metrics also come from the seed distribution, while this example contains arbitrary submissions.

¹Claude Opus 5/4 — see the Declaration on the Use of Artificial Intelligence, p. V.

7.2 Risk Ranking Output and Interpretation Guidance

Each result contains an inactivity-risk score (0–100), an action level (monitor, review, or replace_candidate), prediction regime, and evidence support. Evidence support combines signal completeness, in-distribution fit, and calibration support (Appendix A.8.1). A separate field contains the OpenSSF-derived security-posture score, which the inactivity model does not read (Section 4.1). Figure 7.1 shows risk buckets and action levels. They use separate thresholds on the same score. The review cut is ≥ 50 , below the ≥ 60 high-risk cut, so one medium-risk repository is raised for review. Six of 30 repositories exceed monitor. Table 7.1 lists these repositories and the full-history item; Appendix A.14 gives all 30. Every score is distinct because Section 5.3 replaces flat bin lookup with interpolation.

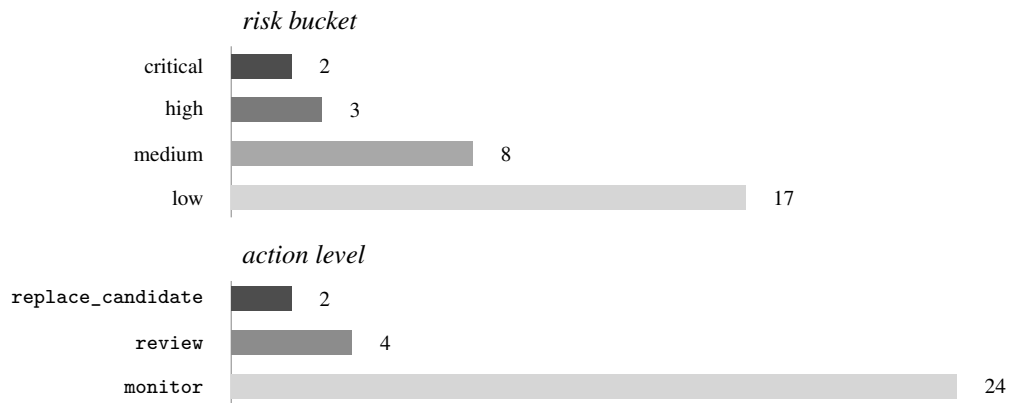


Figure 7.1: Risk buckets and action levels of the 30 dependency repositories of ArchipelagoMW/Archipelago. Each panel totals 30, and darker bars mark the more urgent categories. The two classifications are separate thresholds on the same inactivity-risk score, which is why the six repositories raised above monitor do not coincide with the five in the two highest risk buckets. The review cut (≥ 50) sits below the high-risk cut (≥ 60), so one medium-risk repository is also raised for review.

Comparison of inactivity and security posture. Inactivity and security posture are computed separately and shown without a combined score. `bsdifff4` has inactivity risk 92.1 and posture 6; `flask` has risk 9.5 and posture 77. In the middle, `jinja2` and `markupsafe` receive risks 77.1 and 74.2, but posture scores 58 and 54. They are mature Pallets projects with branch protection and review requirements whose maintenance has slowed. `cymem` combines risk 74.9 with posture 24. The two columns distinguish quieter, well-governed projects from one that is both quiet and weakly governed.

This presentation provides no combination rule or fitted weighting, and whether the two columns improve analyst decisions remains to be tested in the user study proposed in Chapter 9.

Table 7.1: Flagged dependency repositories of ArchipelagoMW/Archipelago (the two `replace_candidate` items and the four `review` items), with the single full-history repository shown for contrast. Risk is the 0–100 inactivity-risk score, and “Supp.” is the per-prediction evidence support. Sec. is the 0–100 security-posture score derived from the repository’s OpenSSF Scorecard record; no part of the inactivity model reads it (Section 4.1). It is shown as — where no Scorecard record was available. “Regime” is full-history (FH) or cold-start (CS). The complete 30-repository ranking is in Appendix A.14.

Package	Source repository	Risk	Action	Supp.	Sec.	Regime
<code>bsdifff4</code>	<code>ilanschnell/bsdifff4</code>	92.1	<code>replace_candidate</code>	0.69	6	CS
<code>setproctitle</code>	<code>dvarrazzo/py-setproctitle</code>	80.4	<code>replace_candidate</code>	0.61	11	CS
<code>jinja2</code>	<code>pallets/jinja</code>	77.1	<code>review</code>	0.77	58	CS
<code>cymem</code>	<code>explosion/cymem</code>	74.9	<code>review</code>	0.75	24	CS
<code>markupsafe</code>	<code>pallets/markupsafe</code>	74.2	<code>review</code>	0.75	54	CS
<code>pymem</code>	<code>srounet/Pymem</code>	54.3	<code>review</code>	0.47	—	CS
<code>flask</code>	<code>pallets/flask</code>	9.5	<code>monitor</code>	0.91	77	FH

Highest-ranked repository. `bsdifff4` (`ilanschnell/bsdifff4`) is a small binary-diff library with risk 92.1. It has low popularity, one maintainer, and no detected release cadence. The cold-start score has evidence support 0.69. These caveats make the repository a candidate for manual review, not a judgement of abandonment (Appendix A.14).

Construct-validity case: Jinja2 and MarkupSafe. `jinja2` (`pallets/jinja`, 77.1) and `markupsafe` (`pallets/markupsafe`, 74.2) are widely used Pallets projects, making their review status initially appear erroneous. At the run date, their last pushes were 2025-06-14 and 2025-09-27. The inactivity label measures the absence of public maintenance signals (Section 4.3), so these scores match the target. No twelve-month outcome is yet available to test them because their windows do not close until 2027. Both have 100% signal completeness, excluding imputation as the cause.

A stable, feature-complete library can resemble a project in decline: both show infrequent commits, long release gaps, and few active contributors. Public metadata cannot distinguish low activity from low *need* for activity (Section 2.3). The author’s inspection found no maintenance concern requiring action in either case, although reviewer decisions were not tested. The scores are consistent with the operational label but not with this manual assessment. They illustrate the construct-validity threat and the need for human review (Section 8.1).²

Low-risk comparison. `flask` (`pallets/flask`) receives risk 9.5 with evidence support 0.91. As the only full-history repository, it has complete coverage and no imputation; its ensemble

²Claude Opus 5/8 — see the Declaration on the Use of Artificial Intelligence, p. V.

members predict 5.4% and 13.5% inactivity risk.

The difference from `jinja2` and `markupsafe` is not caused by regime alone. `flask` was last pushed on 2026-06-10, one month before the run, while the other gaps were thirteen and nine months. Regime and activity are confounded, so this example does not isolate the value of history; Section 6.1.2 does. Evidence support states how much repository information was available beyond the risk score.³

The score is intended to rank repositories for review; it does not certify security or identify a known defect. Low evidence support and imputation reduce confidence. The cold-start model uses fewer features and is calibrated against the sampled inactive rate of 0.661, which exceeds that of a real dependency population (Section 4.12). Its absolute probabilities are therefore overstated until recalibrated for the target population, although the ranking remains useful for triage.

The ranking raises six of the 30 repositories above the monitor level, with `flask` as the full-history reference. Manual inspection found no maintenance concern for the two review items `jinja2` and `markupsafe`, showing why review remains necessary. Whether the ordering improves analyst throughput or decision quality requires a controlled comparison with unranked triage (Chapter 9).

³Claude Opus 5/8 — see the Declaration on the Use of Artificial Intelligence, p. V.

8 Threats to Validity

Threats follow the construct, internal, external, and conclusion validity categories of Wohlin et al. [36]. The intrinsic data and design limitations are summarized in Section 4.12.

8.1 Construct Validity

Repository inactivity is the inverse of the future-window maintenance rule $(t, t + 12 \text{ months}]$ (Section 4.3). It is a proxy: stable projects can need little activity without being abandoned, and development may move to other platforms or release channels. The label therefore cannot prove abandonment. The two-of-four thresholds are heuristic, and merged pull requests proxy maintainer interaction. Requiring sustained activity reduces incidental labels but does not remove this threat.

The sensitivity analysis re-labels all 50,000 snapshots with one-of-four and three-of-four variants (Section 6.4). The inactive rate ranges from 0.581 to 0.746, while ROC-AUC remains within 0.958–0.961. The result is not specific to the two-of-four threshold, but the analysis cannot establish that public activity is the correct maintenance construct. Individual cut-off sweeps remain future work (Chapter 9). The bot-filter ablation additionally shows that excluding bots removes a one-directional bias that masks inactivity (Appendix A.16).

8.2 Internal Validity

Temporal leakage is controlled by using timestamps $\leq t$ for features, the future window for labels, leaving incomplete windows unlabeled, and restricting live enrichment to stable metadata (Section 4.9). Activity persistence remains a separate threat. A single recency signal nearly matches the model on the full test set because many already-quiet repositories stay quiet (Section 6.1.1). On active-at- t , the model exceeds the best trivial rule by 0.061 ROC-AUC (Section 6.3.3). The predictive contribution is claimed from this subset, not the aggregate metric.

Quarterly observations create autocorrelated snapshots. Equal cohort sizes make the test partition the final date, so every test repository also occurs earlier in training (Section 4.6). A repository-disjoint split confines each repository to one partition and refits the model on it (Appendix A.9.2). ROC-AUC decreases only marginally, bounding optimism from recurrence and indicating discrimination on repositories held out from the same frame (Section 6.3.2).

The split is chronological but not embargoed (Section 4.6). Training and validation labels close

after the test observation date, and the boundary between training and validation falls inside the 2024-10-01 cohort. Neither admits information about a test row into its own features or label, so the reported metric is not inflated by leakage across the boundary at t . What they do restrict is the interpretation: the design is a retrospective chronological evaluation with overlapping outcome windows, so it does not demonstrate that the model could have been fitted at the simulated prediction date.

8.3 External Validity

All labels and features come from GitHub events (Section 1.3). The results do not cover GitLab, Codeberg, self-hosted forges, or workflows based on mailing lists, such as the Linux kernel. Platform choice may covary with project age, size, governance, and maintenance, so the direction of this bias is unknown.¹ GitHub samples are also unrepresentative of software development generally: most hosted repositories are personal and inactive, and roughly half of registered users have no public activity [20].²

The foundation seed further requires a license and at least 100 stars. Results do not generalize to very small, private, new, or low-visibility repositories; the “low popularity” slice is relative to this frame (Section 6.3.3). Oversampling dormant and archived projects also makes probabilities and precision conditional on sampled prevalence. Deployment requires recalibration (Section 4.12).

The frame is also sampled retrospectively, which is the more serious of the two sampling threats (Section 4.4). The seed was drawn on 2026-06-11 using push recency and archive state at that date, so bucket membership is decided by behavior falling inside the outcome windows being predicted. The dormant bucket is inactive at a rate of 1.000 in the test cohort, and roughly half of each cohort comes from buckets whose defining condition largely fixes the outcome. Aggregate discrimination is consequently measured on a population that is easier to separate than one sampled at t , and the aggregate figures should be read as conditional on this frame rather than as an estimate of deployment performance. The active-at- t subset is less exposed, since it excludes the already-quiet rows through which the buckets are most legible, but it is drawn from the same frame and inherits the restriction. Correcting this requires a frame built from repository state at each observation date (Chapter 9), which the reported run does not do.³

Finally, the held-out split tests temporal generalization within the dataset, not a fresh sample. Out-of-foundation validation remains future work (Chapter 9).

¹Claude Opus 5/1 — see the Declaration on the Use of Artificial Intelligence, p. V.

²Claude Opus 5/10 — see the Declaration on the Use of Artificial Intelligence, p. V.

³Claude Opus 5/11 — see the Declaration on the Use of Artificial Intelligence, p. V.

8.4 Conclusion Validity

Most metrics are point estimates from one time-aware held-out split. The active-at- t margin additionally has a 95% repository-clustered bootstrap interval excluding zero (Chapter 6), and Section 6.4 tests label variants. Precision remains conditional on sampled prevalence.

ROC-AUC is the primary ranking metric and is insensitive to class prevalence. Average precision is additionally reported for the early-warning subset, whose inactive rate is 0.225; it focuses on the positive class and uses that rate as its no-skill reference [24]. It supports the same conclusion while showing a larger margin over the trivial rules (Section 6.1.3). Two limitations remain. No precision-recall *curve* is plotted, so the shape of the trade-off at a given recall is not shown, only its area. And precision and recall at a single operating point are still reported only at the F1-selected threshold, which is chosen on validation data and is not optimal for any particular triage budget. Qualitative case review is formative support, not confirmatory evidence.

9 Conclusion and Outlook

This thesis examined 12-month OSS dependency inactivity prediction from public repository health and maintenance indicators, together with parallel security-practice context for triage. On the time-aware held-out split, full-history XGBoost reaches ROC-AUC 0.958, Brier 0.078, and expected calibration error 0.035. ROC-AUC remains 0.950 under a repository-disjoint split, bounding optimism from repository recurrence.

The aggregate result mostly reflects recency. The last-commit rule reaches 0.957 on its 4,556 defined rows; XGBoost reaches 0.974 on the same rows (Section 6.1.1). The full test set has inactive rate 0.661 and contains many repositories already quiet at t . On active-at- t , with inactive rate 0.225, XGBoost reaches 0.905 against 0.844 for the best trivial rule. The margin is 0.061, compared with 0.016 on recency-defined rows.

Public repository signals therefore rank 12-month inactivity accurately within the examined dataset, but recency provides most aggregate predictive content. Combining signals adds a modest, measurable improvement for repositories still active at the observation date.

Performance falls to 0.846 on low-visibility repositories, and recall on active-at- t is 0.49 at the selected threshold. Probabilities are calibrated to sampled prevalence. The score therefore prioritizes review and reports per-prediction evidence support; it does not determine abandonment. Security-practice signals remain separate context and are excluded from the inactivity model (Section 3.3). The practical contribution is the per-repository ranked view in Chapter 7.

9.1 Outlook and Future Work

The remaining work is:

- **External validation.** The time-aware split remains inside the foundation dataset (Section 4.11). Evaluation on a fresh repository sample would test generalization beyond its prevalence and seed buckets.
- **A frame sampled at the observation date.** The foundation seed was drawn from repository state at generation time, so entry into the frame depends on behavior inside the outcome windows and all reported metrics are conditional on that construction (Sections 4.4 and 8.3). Building the frame from a repository listing that predates each observation date, so that inclusion uses only information available at t , would remove the dependency and give an aggregate figure comparable to deployment.

- **Systematic qualitative error analysis.** Active-at- t and the trivial baselines isolate predictive skill from persistence (Sections 6.3.3 and 6.1.1). Larger-scale case review is still needed to find predictions made for the wrong reason.
- **Evaluation with analysts.** The demonstrator separates a small candidate set and shows divergent inactivity and security-posture signals, but it does not establish better analyst decisions or throughput (Chapter 7). A controlled practitioner study should compare review time, agreement, and decision quality with and without the ranking and parallel posture column. Public repository data alone cannot answer this question.
- **Additional model families.** More expressive deep-learning architectures and sequence models over repository event history remain untested.
- **Ensemble evaluation.** Runtime averages the staged models of a regime, while reported metrics cover individual artifacts (Section 6.1). Evaluation should compare the ensemble with its strongest member and test model-agreement spread as an uncertainty signal.
- **Prevalence-aware calibration.** The seed oversamples dormant and archived repositories, so probabilities reflect sampled prevalence (Section 4.12). Recalibration to deployment prevalence is required for live probabilities.
- **Extended label sensitivity.** Section 6.4 varies two-of-four to one-of-four and three-of-four. Individual active-month, contributor, merged-pull-request, and release thresholds remain to be swept. The mature libraries in Section 7.2 show the underlying construct problem: public signals may not distinguish stable maturity from decline (Section 8.1).¹
- **Cold-start coverage and on-demand history reconstruction.** Cold-start applies outside the seed, uses one live snapshot, and omits historical signals, including annual commit distribution. Its metrics are measured on seed repositories but applied to arbitrary submissions, so practical performance may be optimistic; evidence support marks such cases. Recent history could be reconstructed through the GitHub REST API or GH Archive’s public BigQuery dataset. The former offers weekly commit and contributor statistics; the latter avoids per-repository rate limits but adds an external query dependency. Neither supplies the full feature set within one request. First-response latency requires prohibitively many issue timelines, and year-over-year features require a second year. Imputing this missing tail into full-history would introduce distribution shift. A third *warm-start* model should instead use only reliably reconstructed live-API features, be validated against the seed cache, and use a rate-limit-aware request cache. This requires further engineering and validation.

¹Claude Opus 5/8 — see the Declaration on the Use of Artificial Intelligence, p. V.

Bibliography

- [1] G. A. A. Prana et al., “Out of sight, out of mind? how vulnerable dependencies affect open-source projects,” *Empirical Software Engineering*, vol. 26, no. 4, pp. 1–34, 2021. DOI: 10.1007/s10664-021-09959-3
- [2] C. Miller, M. Jahanshahi, A. Mockus, B. Vasilescu, and C. Kästner, “Understanding the response to open-source dependency abandonment in the npm ecosystem,” in *Proc. IEEE/ACM 47th International Conference on Software Engineering*, 2025, pp. 2355–2367. DOI: 10.1109/ICSE55347.2025.00004
- [3] Open Source Initiative, *The open source definition*, Accessed: 2026-02, 2026. [Online]. Available: <https://opensource.org/osd>
- [4] M. Souppaya, K. Scarfone, and D. Dodson, “Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities,” National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-218, Feb. 2022. DOI: 10.6028/NIST.SP.800-218
- [5] SLSA Community, *SLSA specification v1.2*, Nov. 2025. [Online]. Available: <https://slsa.dev/spec/v1.2/>
- [6] C. Miller, C. Kästner, and B. Vasilescu, ““we feel like we’re winging it:” a study on navigating open-source dependency abandonment,” in *Proc. ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 1281–1293. DOI: 10.1145/3611643.3616293
- [7] npm, Inc., *kik, left-pad, and npm*, Mar. 2016. [Online]. Available: <https://blog.npmjs.org/post/141577284765/kik-left-pad-and-npm>
- [8] Equifax, *Equifax releases details on cybersecurity incident*, Sep. 2017. [Online]. Available: <https://investor.equifax.com/news-events/press-releases/detail/237/equifax-releases-details-on-cybersecurity-incident>
- [9] Federal Trade Commission, *Equifax to pay \$575 million as part of settlement with ftc, cfpb, and states related to 2017 data breach*, Jul. 2019. [Online]. Available: <https://www.ftc.gov/news-events/news/press-releases/2019/07/equifax-pay-575-million-part-settlement-ftc-cfpb-states-related-2017-data-breach>
- [10] Open Source Security Foundation and OpenJS Foundation, *Open source security and openjs foundations issue alert for social engineering takeovers of open source projects*, Apr. 2024. [Online]. Available: <https://www.linuxfoundation.org/blog/openssf->

openjs-alert-for-social-engineering-takeovers-of-open-source-projects

- [11] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [12] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. DOI: 10.2753/MIS0742-1222240302
- [13] GitHub, *REST API endpoints for licenses*, Accessed: 2026-02, 2026. [Online]. Available: <https://docs.github.com/en/rest/licenses/licenses>
- [14] D. Robinson, K. Enns, N. Koulekar, and M. Sihag, “Two approaches to survival analysis of open source Python projects,” in *Proc. 30th IEEE/ACM Int. Conf. on Program Comprehension (ICPC)*, ACM, 2022, pp. 660–669. DOI: 10.1145/3524610.3527871
- [15] K. A. Hasan, M. Macedo, Y. Tian, B. Adams, and S. H. H. Ding, “Understanding the time to first response in GitHub pull requests,” in *Proc. IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, 2023, pp. 1–11.
- [16] Y. Yehudi, C. Goble, and C. Jay, “Individual context-free online community health indicators fail to identify open source software sustainability,” *arXiv preprint*, 2023. arXiv: 2309.12120 [cs.SE]. [Online]. Available: <https://arxiv.org/abs/2309.12120>
- [17] S. P. Goggins, M. Germonprez, and K. Lumbard, “Making open source project health transparent,” *Computer*, vol. 54, no. 8, pp. 104–111, 2021. DOI: 10.1109/MC.2021.3084015
- [18] CHAOSS Community, *Metrics model: Starter project health*, Accessed: 2026-02, 2026. [Online]. Available: <https://chaoss.community/kb/metrics-model-starter-project-health/>
- [19] N. Zahan, P. Kanakiya, B. Hambleton, S. Shohan, and L. Williams, “OpenSSF scorecard: On the path toward ecosystem-wide automated security metrics,” *IEEE Security & Privacy*, vol. 21, no. 6, pp. 76–88, 2023. DOI: 10.1109/MSEC.2023.3279773
- [20] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, “An in-depth study of the promises and perils of mining GitHub,” *Empirical Software Engineering*, vol. 21, no. 5, pp. 2035–2071, 2016. DOI: 10.1007/s10664-015-9393-5
- [21] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: With Applications in R*, 2nd ed. Springer, 2021. DOI: 10.1007/978-1-0716-1418-1
- [22] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324
- [23] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001. DOI: 10.1214/aos/1013203451

- [24] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” *PLOS ONE*, vol. 10, no. 3, e0118432, 2015. DOI: 10.1371/journal.pone.0118432
- [25] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in *Proc. 22nd International Conference on Machine Learning*, 2005, pp. 625–632. DOI: 10.1145/1102351.1102430
- [26] J. Coelho, M. T. Valente, L. L. Silva, and E. Shihab, “Identifying unmaintained projects in GitHub,” in *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2018, 15:1–15:10. DOI: 10.1145/3239235.3240501
- [27] J. Coelho, M. T. Valente, L. Milen, and L. L. Silva, “Is this GitHub project maintained? measuring the level of maintenance activity of open-source projects,” *Information and Software Technology*, vol. 122, p. 106274, 2020. DOI: 10.1016/j.infsof.2020.106274
- [28] X. Li, S. Moreschini, F. Pecorelli, and D. Taibi, “OSSARA: Abandonment risk assessment for embedded open source components,” *IEEE Software*, vol. 39, no. 4, pp. 48–53, 2022. DOI: 10.1109/MS.2022.3163011
- [29] E. Krause, *OSS Risk Radar*, Source repository, pinned at commit f5e688e, which contains the reported dataset, staged model artifacts, and training notebook. Accessed: 2026-07, 2026. [Online]. Available: <https://github.com/esuarkeN/oss-risk-radar/tree/f5e688e>
- [30] GH Archive, *GH Archive: GitHub archive data*, 2026. [Online]. Available: <https://www.gharchive.org/>
- [31] E. Krause, *Historical dataset builder*, Pinned at commit f5e688e. Accessed: 2026-07, 2026. [Online]. Available: <https://github.com/esuarkeN/oss-risk-radar/blob/f5e688e/docs/methodology/historical-dataset-builder.md>
- [32] deps.dev, *deps.dev API*, Accessed: 2026-02, 2026. [Online]. Available: <https://docs.deps.dev/api/v3/>
- [33] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, ACM, 2016, pp. 785–794. DOI: 10.1145/2939672.2939785
- [34] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?” In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 35, 2022, pp. 507–520.
- [35] OSS Review Toolkit, *Analyzer*, Accessed: 2026-02, 2026. [Online]. Available: <https://oss-review-toolkit.github.io/ort/docs/tools/analyzer>

- [36] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer, 2012. DOI: 10.1007/978-3-642-29044-2
- [37] Stack Overflow. “Technology | 2025 stack overflow developer survey.” Accessed 2026-06-04. [Online]. Available: <https://survey.stackoverflow.co/2025/technology>
- [38] GitHub. “Octoverse: The state of open source.” Accessed 2026-06-04. [Online]. Available: <https://octoverse.github.com/>

Declaration on the Use of Artificial Intelligence

In this thesis, generative AI systems were used in a supporting role for editorial, technical, and methodological tasks. These included researching potentially relevant literature, the linguistic and formal revision of text passages, the drafting and revision of individual sections against reviewer feedback, support in the development and documentation of software components, and checking the scientific soundness of the work. All content, sources, and changes proposed by AI systems were independently reviewed by the author before being adopted; the research question, the study design, the empirical results, and the conclusions drawn from them are the author’s own.

Table 9.1 documents the individual interactions in the order in which they were used. Each entry gives the system with its version, the opening prompt, and what the output was used for together with the section it entered. Where the same system was used several times, the entries are numbered (for example *Claude Opus 5/2*). Passages in the running text that were drafted with AI support carry a footnote referring to the corresponding entry; a footnote prefixed with “cf.” marks a passage for which the output served only as a starting point.

Table 9.1: Use of generative AI systems in the preparation of this thesis and the software artifact, listing the system and version, the opening prompt, and how and where the results were used.

System	Opening prompt	Use of results and location
GPT-5.5 (Deep Research and Thinking)/1	“Hey i write my bachelorthesis about predicting OSS inactivity risk. can you find me sources about that topic and also I found these metrics https://www.chaoss.community/kb-metrics-and-metrics-models/ and wanted to use them can you find sources that actually support those metrics. ty”	One interaction serving two purposes: research of scientific sources on how the inactivity or activity state of open-source projects can be determined, and a search for literature supporting the CHAOSS metrics. All proposed sources were reviewed for scientific quality and thematic relevance before a subset was adopted; the CHAOSS material was checked against the project’s own documentation. <i>Chapters 2 and 3; Sections 2.2 and 4.7; bibliography.</i>

continued on next page

Table 9.1 — continued from previous page

System	Opening prompt	Use of results and location
Codex with GPT-5.5/1	“Hey bud can you give me an powershell script which I can use to download automatically a list of csv repos such as follows for input given timestamps from github archive, ty.”	Development of a script for automatically downloading relevant GH Archive data for a structured set of GitHub repositories listed in a reference CSV file. The generated script was reviewed, adapted, and used for automated data collection. <i>Software artifact; described in Section 5.2.</i>
Codex with GPT-5.5/2	“so we have our Regression and xgboost model but I want to also tesxt a neural net if it performs better so we are scientifically save for that case and show its better or not can you help me setup a plan on how we would impelemtn that.”	Analysis of possible extensions of the existing prediction model with a neural network, and proposals for integrating a corresponding model component into the existing software architecture. The proposed approaches were evaluated and used as the basis for implementing an additional neural model component. <i>Software artifact; reported in Section 6.1.</i>
Codex with GPT-5.5/3	“wouldnt be a full migration to a jupyter notebook smart as its a thesis level program and the python scripts remain the same but we have better overview and use the artifact of the model in the end anyway. create a plan for that” (2026-06-04)	Transfer and documentation of the existing Python scripts into a Jupyter notebook, with an assessment of whether the code should be fully transferred into the notebook or included via the existing scripts. A notebook was created that references the existing scripts and documents the individual steps of data processing and model evaluation. <i>Software artifact; described in Section 5.4 and Appendix A.7.6.</i>
Claude 4.8/1	Opus “so now check the whole tex part again for anything vague or not backed by sources and then we fix it together so you can ask me questions then” (2026-07-09)	Review of the L ^A T _E X files with regard to scientific style, argumentative traceability, and whether external statements are supported by sources and empirical statements by the author’s own findings. The identified issues were reviewed and corrected where necessary; the final assessment and adoption of the changes was carried out independently. <i>Whole thesis; no new content generated.</i>

continued on next page

Table 9.1 — continued from previous page

System	Opening prompt	Use of results and location
Claude 4.8/2	Opus “okay and another thing well yeah I have 60 pages the limit is around 25 to 30 yeah so I have to well shorten and cut stuff but also I feel like we need a lot of it but yeah in the end its a scientific work so what can we actually move to the appendix (rename additional content to appendix pls) and move out so we actually have the scientifically relevant infos all in the main chapters” (2026-07-02)	Assessment of which extensive tables, feature descriptions, and supplementary content could be moved from the main body to the appendix to improve readability and length. Suitable detailed information was moved; the summaries and key statements required for understanding the methodology and results remained in the main body. <i>Appendix A.3 and A.6; structural only.</i>
Claude 4.8/3	Opus “yeah we should link our tables and references to the appendix from the main text aswell as back from appendix to the main text where you probably jumped to the appendix. Do you understand what i mean? if not pls ask me again so you dont assume, ty” (2026-07-10)	Creation and review of cross-references between the content moved to the appendix and the corresponding sections of the main text. The proposed cross-references were implemented. <i>Whole thesis; cross-references only.</i>
Claude 4.8/4	Opus “okay i asked another ai for some feedback and this is what we got, is it right and well do you agree and can we fix that? its in feedback.md” (2026-07-23)	Final review of the thesis for L ^A T _E X errors, scientific style, consistency of argumentation, and the scientific soundness of the chosen methodology. Minor formal and technical adjustments were made; content-related suggestions were independently verified before being adopted. <i>Whole thesis; no new content generated.</i>
Claude Opus 5/1	“so i got feedback under downloads, thesisfeedback. WE should implement it and well yeah check for the other informations i got, like the other pdf and ppt files from yesterday regarding bachelor sutff so well we should chekc against taht again if everythign is sufficient here aswell” (2026-07-27)	Extraction of the annotations from a reviewer-commented PDF of the thesis and revision of the affected passages: reworking the motivation to lead with vulnerability exposure, adding the GitHub-only scope restriction, defining the term <i>demonstrator</i> , naming the security-practice signals, and unifying the label terminology. Drafted with AI support and reviewed by the author. <i>Sections 1.1, 1.2, 1.3, 4.1, 4.3, 4.4, 4.5, 7.2, 8.3.</i>

continued on next page

Table 9.1 — continued from previous page

System	Opening prompt	Use of results and location
Claude Opus 5/2	“actually i get more and more the feeling i have more like a master thesis rather than a bachelor thesis. As well im diving very deep into stuff but not explain certain basics like AUROC in the basics or even in glossary” (2026-07-27)	Drafting of a plain-language explanation of the evaluation metrics (discrimination versus calibration, ROC-AUC, Brier score, expected calibration error, held-out test set) for readers without a machine-learning background, and clarification of why the recency baseline is a like-for-like comparison. Drafted with AI support and reviewed by the author. <i>Section 2.4; abstract.</i>
Claude Opus 5/3	Same originating request as <i>Claude Opus 5/2</i> ; the glossary was produced from the part of it concerning undefined basics (“... not explain certain basics like AUROC in the basics or even in glossary”, 2026-07-27).	Drafting of a glossary of the recurring technical terms of the thesis, distinguishing established machine-learning terminology from the definitions introduced by this work. Drafted with AI support; the entries were checked against the corresponding sections by the author. <i>Glossary (front matter).</i>
Claude Opus 5/4	“now we got the full comments” (2026-07-27), referring to a second reviewer-annotated PDF of the thesis; preceded by “yes do that pls” on a partial set of the same reviewer’s comments.	Revision of the methodology, implementation, and results chapters against a second round of reviewer comments: making the temporal structure of the observation snapshots explicit, justifying the inverted maintenance variable and the binned calibrator, motivating the slice evaluation and the repository-disjoint split, defining the foundation seed and the human-actor filter, and explaining what a <i>reconstructed proxy</i> means. Drafted with AI support and reviewed by the author. <i>Sections 4.3, 4.6, 4.7, 4.8, 5.1, 5.3, 6.1.1, 6.1.2, 7.1; Appendix A.16; Chapter 8.</i>
Claude Opus 5/5	Same originating request as <i>Claude Opus 5/4</i> ; the figure answers the reviewer comment “diagram would be great for stuff like this. spart auch viel text”.	Generation of a TikZ bar chart of the standardized logistic-regression coefficients, replacing a prose enumeration of the same values. The plotted values were taken from the author’s own training run and verified against it. <i>Figure 6.1, Section 6.3.1.</i>

continued on next page

Table 9.1 — continued from previous page

System	Opening prompt	Use of results and location
Claude Opus 5/6	Same originating request as <i>Claude Opus 5/4</i> ; the check was triggered by the reviewer comment “das hast du glaub ich rausgenommen oder?” on the package-derived features.	Verification of which model inputs are constant in the reported dataset, and drafting of the resulting disclosure. The check was run against the committed training data and the model run artifact; the finding that ten of the sixty inputs carry no repository-level signal — eight constant across all rows and two varying only with the observation date — was confirmed by the author. <i>Section 4.7; Appendix A.5.</i>
Claude Opus 5/7	“yeah go ahead” (2026-07-27), authorising the adaptation of typeface, margins, page numbering and front-matter order to the Proবাদis <i>Anforderungen</i> ; the bibliography page ranges were checked during the request recorded as <i>Claude Opus 5/1</i> .	Verification of the missing page ranges of five published bibliography entries against DBLP and the publishers’ records, and adaptation of the document formatting to the Proবাদis requirements (typeface, margins, page numbering, order of the front matter). <i>Bibliography; document formatting only, no content generated.</i>
Claude Opus 5/8	“You are acting as a strict academic language editor for an English-language bachelor’s thesis in computer science . . . start with the introduction” (2026-07-28), a structured editing brief followed by chapter-by-chapter requests over the same session.	Passage-by-passage review of all chapters, the abstract, and the appendix for vague or unsupported wording, overstated certainty, inconsistent terminology, and claims not carried by the reported results. Each finding was presented to the author with the proposed wording; the author decided which to apply, and the approved corrections were then made. No new content, sources, or results were introduced. <i>Whole thesis; wording and consistency only.</i>

continued on next page

Table 9.1 — continued from previous page

System	Opening prompt	Use of results and location
Claude Opus 5/9	Same originating session as <i>Claude Opus 5/8</i> ; the re-verification was requested with “we should update all numbers to our newest run” (2026-07-28).	Re-training of the reported model on the committed dataset, which reproduced the staged run’s held-out metrics to six decimal places, and recomputation of the reported figures from the unrounded output. The recomputation showed that two margins had been derived by subtracting already-rounded values and that two comparisons used different trivial baselines; the affected figures were corrected and the comparators stated explicitly. A reproducible harness for the label-definition sensitivity analysis was added to the software artifact. All figures were produced by executing the author’s own pipeline on the author’s own data. <i>Sections 6.1.1, 6.1.2, 6.3.3, 6.4; Chapter 9; Appendix A.13 and A.12; software artifact.</i>
Claude 5/10	Opus Same originating session as <i>Claude Opus 5/8</i> ; prompted by “for 2 and 3 get the sources and check again then” (2026-07-28).	Verification of numeric claims attributed to cited works against the publications themselves, and of the demonstrator’s characterisations of individual repositories against the public GitHub API. Two statements that the cited sources did not support were removed, vague quantifiers were replaced by the figures the sources report, and one incomplete bibliography entry was completed. The corrected characterisation of two repositories in the demonstrator was verified by the author before adoption. <i>Sections 1.1, 3.1, 7.2; bibliography.</i>

continued on next page

Table 9.1 — continued from previous page

System	Opening prompt	Use of results and location
Claude 5/11	Opus “codex checked and said this should I do to enhance my thesis: ... is that feedback right?” (2026-07-28), pasting a review of the thesis and software artifact produced by another AI system, followed by “yes do it pls” authorising the corrections the review was found to justify.	Verification of an external AI review’s claims against the executable code and the generated dataset, and drafting of the resulting corrections. Four claims were confirmed by direct measurement and three were found to be unsupported; the author decided which to act on. The confirmed items were: the boundary between the training and validation partitions falls inside one observation cohort and their labels close after the test observation date; the foundation seed was drawn from repository state after the outcome windows had closed, with the per-stratum inactive rates recomputed from the committed dataset; the repository-disjoint split refits the model and was described incorrectly as a re-analysis; and two feature inputs vary only with the observation date. All measurements were produced by executing the author’s own pipeline on the author’s own data. <i>Sections 4.4, 4.6, 4.7, 4.9, 5.3, 6.3.2; Chapters 8 and 9; Appendices A.5 and A.9.</i>

continued on next page

Table 9.1 — continued from previous page

System	Opening prompt	Use of results and location
Claude 5/12	Opus Same originating session as <i>Claude Opus 5/11</i> ; the analysis was requested with “do item d” (2026-07-28), referring to the one item of the external review that proposed an additional measurement rather than a correction.	Addition of an average-precision evaluation of the early-warning subset, which the thesis had previously recorded as a missing metric. A reproducible harness was added to the software artifact; it aborts unless the re-scored predictions reproduce the reported held-out ROC-AUC, and it reproduced the previously reported subset figures exactly. The metric definition, the reported areas, the bootstrap intervals, and the accompanying discussion were drafted with AI support and reviewed by the author. All figures were produced by executing the author’s own pipeline on the author’s own data. <i>Sections 4.11, 6.1.3; Chapter 8; Appendix A.11; glossary; software artifact.</i>

A Appendix

This appendix provides code and data availability, feature and method reference material, implementation and reproducibility details, and supplementary evaluation output. Its sections cover code availability (Appendix A.1), health indicators (Appendix A.2), implementation detail (Appendix A.6), evaluation instruments (Appendix A.9), and undefined-feature handling (Section A.5) for the implementation described in Chapter 5.

A.1 Code and Data Availability

The artifact of this thesis — the historical dataset builder, the feature and label computation, the model implementations, the training notebook, the scoring service, and the orchestration and analysis scripts — is publicly available at

`https://github.com/esuarkeN/oss-risk-radar`

The results reported here were produced from the pinned revision f5e688e [29], which carries the reported dataset, the staged model artifacts, and the training notebook in the state they were evaluated in. That revision is what the dataset hash in Appendix A.12 identifies, and all file paths given in this thesis are relative to the repository root. The commands that regenerate the foundation seed, rebuild the dataset, and retrain the models are listed in Appendix A.12, together with the per-run documentation items. Analyses added after the pinned revision — among them the label-definition sensitivity harness of Appendix A.13 — are available on the default branch rather than in f5e688e.

A.2 Observable Health Indicator Categories

Table A.1 lists the observable health categories, example signals, public sources, and expected relevance (Section 2.2). Section 4.7 implements the corresponding feature groups, and Appendix A.3 defines each feature. Security-practice indicators remain parallel triage context, not inactivity-model inputs (Section 3.3).

A.3 Full-History Feature Reference

This section expands the feature engineering step of Section 4.7.

Table A.1: Categories of observable health indicators and their expected relevance

Category	Example signals	Public source	Expected relevance
Commit activity	Commits over recent windows; active commit months; days since last commit	GH Archive push events	Direct evidence of ongoing development
Contributor activity	Recent and new contributors; contributor concentration	GH Archive commit actors	Team breadth and single-maintainer (bus-factor) risk
Issue and PR activity	Opened/closed issues; PR merge ratio; response and merge latency	GH Archive issue/PR events	Maintenance interaction and responsiveness
Release activity	Releases; days since last release; versions published	GH Archive releases and package registry	Whether usable versions still reach users
Age and popularity	Repository and package age; stars; forks; popularity tier	GitHub metadata and package registry	Maturity and reach context
Security practice	Selected OpenSSF Scorecard checks	OpenSSF Scorecard	Parallel security-posture context, not an inactivity predictor

The full-history model consumes a feature vector of 60 variables under feature set `feature-set-v4-full-history`. These come from two sources, documented separately below: 43 *reconstructed historical* features, derived from the GH Archive event history before t and summarized by group in Table A.2 and individually in Table A.3; and 17 *current-snapshot* features, obtainable from a single observation of the repository and package metadata, listed in Table A.4. The cold-start regime uses exactly the 17 current-snapshot features and none of the historical ones, which is what makes it applicable to repositories with no reconstructed history. Table A.3 additionally lists two variables that are computed and exported for inspection but are *not* part of the 60-variable model vector: `repo_archived_at_obs`, excluded so that archival cannot become a shortcut label, and `repo_age_days`, which is identically zero under the reproducible offline build because that build carries no repository creation date (Section 4.4). Because the feature names are compact identifiers whose exact meaning is not obvious to a reader who is new to the artifact, this appendix documents each feature individually. The identifiers used here are the same strings that appear in the training code, in the exported model artifacts, and in the runtime interface, so the tables double as a shared glossary across the thesis and the software.

Four conventions apply throughout and explain the recurring suffixes in the names. First, a numeric suffix denotes a trailing time window that ends at the observation time t : `_30d`, `_90d`, and `_365d` count events in the 30, 90, and 365 days immediately before t , and `_at_obs` denotes a state measured exactly at t . Second, all commit- and contributor-based counts use a human-actor filter, so activity generated by bots does not inflate the maintenance evidence. Third, wherever

an absolute count would not be comparable across projects of very different sizes, the feature set also provides a relative or trend-based form (a ratio, a share, or a year-over-year change), because a large project and a small project can be equally well or badly maintained at very different absolute volumes. Fourth, a feature that is *undefined* for a repository is distinguished from one that is genuinely zero (Section 4.7): the “days since” features are right-censored at the age of the observable record, and the latency medians are left undefined and imputed by the model artifact rather than being set to zero, which would read as an instantaneous response. Every value is computed from information available at or before t only; the reasoning behind this restriction, and the distinction between it and the legitimate activity-persistence effect, is discussed in Section 4.9.

Table A.2: Implemented full-history feature groups

Feature group	Example features
Dependency context	Dependency count of the resolved package version at the observation time.
Commit activity	Commits in the last 30, 90, and 365 days; active commit months; days since last commit.
Contributor activity	Contributors in the last 90 and 365 days; new contributors; contributor concentration.
Issue activity	Opened and closed issues; issue closure ratio; issue backlog growth; stale open issues.
Pull request activity	Opened, merged, and closed-unmerged pull requests; pull request merge ratio; stale open pull requests; pull-request response and merge latency.
Release activity	Releases in the last 365 days; days since last release; versions published in the last 365 days; recent release flag.
Age and popularity proxies	Package age; repository age (diagnostic only); stars and forks observed in the available historical data; popularity tier.
Derived risk proxies	Activity drop; contributor drop; release gap risk; concentration risk score.
Responsiveness and backlog pressure	Issue first-response time; issue resolution time; stale issue share; pull-request response and merge latency.

Table A.3: Reconstructed historical feature reference (feature-set-v4-full-history): exact definition and rationale for each of the 43 historical model inputs, plus the two diagnostic-only variables `repo_archived_at_obs` and `repo_age_days`.

Feature	Definition (all measured at or before t)	Why it is included
<i>Commit activity</i>		
<code>commits_30d</code>	Number of human commits in the 30 days before t .	Short-term development pulse; captures whether the project is moving right now.
<code>commits_90d</code>	Number of human commits in the 90 days before t .	Quarter-scale activity level that is less noisy than the 30-day count.

continued on next page

Table A.3 continued from previous page

Feature	Definition (all measured at or before t)	Why it is included
commits_365d	Number of human commits in the 365 days before t .	Annual development volume; also the baseline against which the year-over-year drop is measured.
active_commit_months_365d	Count of distinct calendar months (0–12) in the last year that contain at least one human commit.	Separates steady maintenance from a single burst: 300 commits in one month is a weaker health signal than the same work spread across ten months.
days_since_last_commit	Days between t and the most recent human commit; right-censored at the age of the observable record when no commit is ever observed.	Direct staleness signal; a long silence is one of the strongest indicators of impending inactivity.
<i>Contributor activity and concentration</i>		
contributors_90d	Distinct human contributors in the last 90 days.	Breadth of recent participation; a proxy for the size of the active team.
contributors_365d	Distinct human contributors in the last 365 days.	Annual contributor base; the reference for the contributor drop.
new_contributors_365d	Contributors active in the last year who never committed before the window started.	Measures inflow of fresh maintainers; a project with no newcomers is more exposed to attrition.
top1_contributor_commit_share_365d	Share (0–1) of the last year’s commits made by the single most active contributor.	Key-person dependency: a high share means the project is fragile if that one person leaves.
top2_contributor_commit_share_365d	Combined commit share of the two most active contributors in the last year.	Detects a very small core team even when no single person dominates.
contributor_concentration_index	Herfindahl index: sum of squared per-contributor commit shares over the last year.	A single scalar for how concentrated contributions are (1.0 = one person, near 0 = evenly spread).
maintainer_concentration_flag	1 if the top contributor share ≥ 0.7 or the top-two share ≥ 0.85 , else 0.	Binary bus-factor alarm for the clearly single-maintainer case.
<i>Issue activity and backlog</i>		
opened_issues_90d	Issues created in the last 90 days.	Inbound user demand and engagement with the project.
closed_issues_90d	Issues closed in the last 90 days.	Maintainer throughput on the issue tracker.
issue_closure_ratio_90d	Closed divided by opened issues over the last 90 days.	Whether maintainers keep pace with inflow, expressed relatively so it is comparable across project sizes.

continued on next page

Table A.3 continued from previous page

Feature	Definition (all measured at or before t)	Why it is included
issue_backlog_growth_90d	Relative change in the open-issue count between $t - 90$ days and t .	A backlog that is trending upward signals that maintainers are losing control; size-normalized.
stale_open_issues_count_at_obs	Issues opened before $t - 90$ days that are still open at t .	Long-unaddressed items indicate neglect of the tracker.
<i>Pull-request activity</i>		
opened_prs_90d	Pull requests created in the last 90 days.	Contribution inflow from outside the core team.
merged_prs_90d	Pull requests merged in the last 90 days.	Direct evidence that maintainers are still integrating outside work.
closed_unmerged_prs_90d	Pull requests closed without merging in the last 90 days.	A high count relative to merges can signal rejection or disengagement from contributions.
pr_merge_ratio_90d	Merged divided by opened pull requests over the last 90 days.	Responsiveness to contributors, expressed relatively.
stale_open_prs_count_at_obs	Pull requests opened before $t - 90$ days that are still open at t .	Ignored contributions discourage further contribution and indicate maintainer absence.
<i>Release activity</i>		
releases_365d	GitHub releases published in the last 365 days.	Shipping cadence: whether usable versions still reach users.
days_since_last_release	Days since the most recent release or package version; right-censored at the age of the observable record when no release is ever observed.	Release staleness; complements commit staleness for projects that ship rarely.
versions_published_365d	Package-registry versions published in the last 365 days.	Package-level shipping, which can differ from repository releases.
<i>Dependency context</i>		
dependency_count_at_obs	Declared dependency count of the resolved package version at t .	Proxy for maintenance burden and integration complexity.
<i>Age and popularity proxies</i>		
package_age_days	Days from the first published package version to t .	Maturity context: young and long-established packages have different baseline inactivity risk.
repo_age_days	Days from repository creation to t .	Diagnostic metadata only, and not a model input: the offline build supplies no creation date, so this column is identically zero (Section 4.4).

continued on next page

Table A.3 continued from previous page

Feature	Definition (all measured at or before t)	Why it is included
stars_total_at_obs	Cumulative stars observed up to t (a lower-bound proxy reconstructed from archive events).	Popularity and visibility; more attention was expected to correlate with continued maintenance, although the association observed in this dataset runs the other way (Section 6.3.1).
forks_total_at_obs	Cumulative forks observed up to t .	Reuse and derivative activity around the project.
popularity_tier_at_obs	Low / medium / high tier (0/1/2) derived from star and fork thresholds.	Coarse popularity bucket, because inactivity dynamics differ between obscure and widely used projects.
<i>Diagnostic and binary flags</i>		
repo_archived_at_obs	1 if the repository is archived by t , else 0.	Diagnostic metadata only, and not a model input; already-archived rows are excluded from fitting so archival cannot become a shortcut label.
has_recent_release_flag	1 if any release or package version appeared in the last 365 days.	Simple “is it still shipping” indicator that is robust when exact counts are sparse.
has_recent_pr_merge_flag	1 if any pull request was merged in the last 90 days.	Simple “are maintainers still integrating” indicator.
has_issue_activity_365d	1 if at least one issue was opened in the last 365 days, else 0.	Makes “no issue tracker activity at all” an explicit signal, so its absence cannot reach the model disguised as a zero-valued response latency.
has_pr_activity_365d	1 if at least one pull request was opened in the last 365 days, else 0.	The pull-request counterpart; distinguishes a project nobody contributes to from one that merges contributions instantly.
<i>Derived risk proxies</i>		
activity_drop_365d_vs_prev_365d	Relative decline in commits from the previous year to the last year (positive = fewer commits).	Captures deceleration, enabling earlier detection of decline than the absolute level alone.
contributors_drop_365d_vs_prev_365d	Relative decline in the contributor count from the previous year to the last year.	Detects a shrinking maintainer base before commits fully stop.
release_gap_risk	Score in $[0,1]$ for how overdue the next release is relative to the project’s own release cadence (age-based when no release history exists).	Normalizes release silence against the project’s normal rhythm rather than an absolute threshold.

continued on next page

Table A.3 continued from previous page

Feature	Definition (all measured at or before t)	Why it is included
concentration_risk_score	Composite score in $[0,1]$ combining top-1 and top-2 shares, the Herfindahl index, and the concentration flag.	A single interpretable bus-factor risk value that consolidates the concentration features.
<i>Responsiveness and backlog pressure</i>		
issue_first_response_median_days_365d	Median days from issue creation to the first maintainer response over the last year; <i>undefined</i> if no issue received a response in the window.	Slowing first responses indicate disengaging maintainers, independently of resolution speed.
issue_resolution_median_days_365d	Median days from issue creation to closure over the last year; <i>undefined</i> if no issue was closed in the window.	How quickly reported problems are actually resolved.
stale_issue_share_at_obs	Stale open issues divided by the current open backlog.	The fraction of the backlog that is old, expressed relatively so it is comparable across sizes.
pr_response_median_days_365d	Median days to the first response, merge, or close on pull requests over the last year; <i>undefined</i> if no pull request was responded to.	Responsiveness to contributors, the pull-request counterpart to issue first response.
pr_merge_latency_median_days_365d	Median days from creation to merge for merged pull requests over the last year; <i>undefined</i> if no pull request was merged.	Integration speed for work that is ultimately accepted.

The remaining 17 model inputs are *current-snapshot* features (Table A.4). They require only a single observation of the repository and its package metadata, with no event history, which is precisely why they constitute the entire cold-start feature set (`feature-set-v4-cold-start`). They are also part of the full-history vector, so a full-history model sees both tables.

A.4 Foundation Seed Composition

The foundation seed is generated from the GitHub search API over public, non-fork repositories with a license object and at least 100 stars, drawn separately from three sampling buckets: active, dormant, and archived repositories. The buckets are sampling provenance only and never enter the supervised label (Section 4.4). Table A.5 contrasts the target frame with the seed as actually built. Two buckets returned fewer repositories than requested — the GitHub search frame simply did not contain enough matching dormant and low-star active projects — and a fallback bucket made up the missing count with active repositories instead. That is why the realized seed is majority-active despite a frame designed to oversample inactivity.

Each of the three sampling groups is drawn from several star-range buckets, which is why the shortfalls are visible only at bucket level. The realized active count of 2,601 comprises 2,095 repositories from four active buckets — of which the active-foundation bucket (100–499

Table A.4: Current-snapshot feature reference: the 17 features obtainable without event history. These form the complete cold-start vector and are also included in the full-history vector.

Feature	Definition (measured at t)	Why it is included
has_repository_mapping	1 if the package resolves to a source repository.	Without a repository, all activity signals are unavailable; the model must know this.
is_direct_dependency	1 if the package is a direct rather than transitive dependency.	Triage context: direct dependencies are more readily replaced.
last_push_age_days	Days since the most recent push to the default branch; right-censored when no push is observed.	The cold-start counterpart to commit staleness.
last_release_age_days	Days since the most recent release; right-censored when none is observed.	Shipping staleness without needing release history.
release_cadence_days	Mean days between consecutive releases; <i>undefined</i> with fewer than two releases.	The project's normal shipping rhythm, against which silence is judged.
recent_contributors_90d	Distinct contributors in the last 90 days.	Breadth of the currently active team.
contributor_concentration	Commit share of the most active contributor.	Key-person dependency without full history.
open_issue_growth_90d	Relative change in the open-issue count over 90 days.	Whether the backlog is trending upward.
pr_response_median_days	Median days to first response on pull requests; <i>undefined</i> if none were responded to.	Responsiveness to contributors.
stars_log1p	$\log(1 + \text{stars})$ at t .	Popularity on a compressed scale; raw counts span several orders of magnitude.
forks_log1p	$\log(1 + \text{forks})$ at t .	Reuse and derivative activity, likewise compressed.
open_issues_log1p	$\log(1 + \text{open issues})$ at t .	Backlog size, compressed and comparable across project sizes.
ecosystem_npm, ecosystem_pypi, ecosystem_go, ecosystem_maven, ecosystem_other	Five mutually exclusive one-hot indicators of the package ecosystem.	Baseline inactivity rates and release conventions differ by ecosystem.

Table A.5: Foundation seed buckets: target frame versus realized seed (5,000 repositories).

Sampling bucket	Target	Realized	Target share	Realized share
Active	2,250	2,601	45.0%	52.0%
Dormant	1,750	1,399	35.0%	28.0%
Archived	1,000	1,000	20.0%	20.0%
Total	5,000	5,000	100%	100%

stars) returned 845 of the 1,000 requested — plus a 506-repository fallback bucket used to reach the target size. The dormant count of 1,399 comprises three buckets, the dormant-foundation bucket returning 649 of 1,000; the three archived buckets returned their full 1,000.¹

A.5 Undefined Feature Handling

This section gives the exact rules summarized in Section 4.7.

Censoring bound. The age of the observable record is the interval from the later of the repository creation date and the start of archive coverage up to the observation time t . When a repository has no observed commit (or no observed release), the corresponding “days since” feature is set to this bound, which is the largest value consistent with the data. When the record begins at or after t — that is, when nothing at all was observable before the observation date — there is no interval over which the event could have been missed and therefore no bound to censor against; the feature is then undefined rather than zero. In the reported dataset this applies to 7,687 of 50,000 rows, whose repositories have no GH Archive event before their observation date.

Undefined values. The four responsiveness medians (`issue_first_response_median_days_365d`, `issue_resolution_median_days_365d`, `pr_response_median_days_365d`, `pr_merge_latency_median_days_365d`) and the two censored age features are exported as absent keys rather than as null or 0, so that no consumer can silently read them as zero. Downstream, the gradient-boosted trees route undefined values natively through XGBoost’s missing-value branch, while the logistic-regression and neural-network models substitute the per-feature training-set median, which is stored in the model artifact alongside the standardization parameters and re-applied identically at inference. Because a substituted median standardizes to approximately zero, an undefined feature contributes little to the linear score rather than pulling it toward the favorable extreme.

Why it matters. Before this correction, an undefined latency was stored as 0.0. Since 0 days is the best possible response time, this encoded the absence of any observed response as an instantaneous one. The affected share is large: each of the four medians is undefined for between 46.8% and 60.4% of the 50,000 snapshots, and 36.7% of snapshots have neither issue nor pull-request activity in the trailing year at all.² The variable was consequently a strong predictor of inactivity with an inverted meaning, and the feature attributions of Section 6.3.1 reflected

¹Claude Opus 5/8 — see the Declaration on the Use of Artificial Intelligence, p. V.

²Claude Opus 5/9 — see the Declaration on the Use of Artificial Intelligence, p. V.

data availability rather than maintenance behavior. The same defect made `days_since_last_commit` non-monotonic, which in turn depressed the trivial recency baseline against which the learned models are compared (Section 6.1.1).

Offline-build consequences. Because the reported dataset is built through the reproducible offline path (Section 4.4), the repository creation date, archival timestamp, and fork flag are never populated, and three consequences follow. First, `repo_age_days` is identically zero and is therefore excluded from the model vector and from any age analysis. Second, `repo_archived_at_obs` is identically zero, so the archival condition of the maintenance rule never fires (Section 4.3) and the rule excluding already-archived rows from fitting removes none. Third, build-time fork exclusion is inert, though the seed is generated with a non-fork filter so forks are already absent. Populating these attributes would require live enrichment of every seed repository and is noted as future work in Chapter 9.

Zero-variance inputs in the reported dataset. A related consequence deserves to be stated explicitly, because it qualifies the feature counts used throughout this thesis. The foundation seed is repository-first: it is drawn from GitHub search rather than from a package registry, and the offline build performs no `deps.dev` lookup. Several package-derived inputs of the feature vector therefore have nothing to vary over, and take the same value in all 50,000 rows. Eight of the 60 model inputs are constant in this way:

- the five ecosystem indicators, since every row carries the same repository-first provenance (`ecosystem_other` is 1, the other four 0);
- `is_direct_dependency` and `has_repository_mapping`, both identically 1, because a seed repository is by construction a resolved, directly named project;
- `dependency_count_at_obs`, identically 0.

Two further inputs vary across the dataset but carry no *between-repository* information, and are counted separately for that reason. Both follow from one mechanism: when no registry version history is available and the build runs offline, the builder records a single synthetic publication event for every repository, dated to the start of the observation period, 2023-01-01. `package_age_days` is then the distance from that date to the observation date and takes exactly ten distinct values — 0, 90, 181, ..., 821 — one per quarterly cohort of 5,000 rows. `versions_published_365d` counts publications in the trailing year and is therefore 1 for the four 2023 cohorts and 0 for the six later ones. Neither is a property of the repository; each is a function of the observation date alone. Ten of the sixty model inputs consequently carry no repository-level signal: the eight constants above and these two. The effective feature vector of the reported run is closer to 50 informative inputs than to 60. Seven of the eight constants — the five ecosystem indicators, `is_`

`direct_dependency`, and `has_repository_mapping` — belong to the current-snapshot set, and neither cohort-dependent input does, so the cold-start vector is correspondingly closer to 10 than to 17.

A third input is affected indirectly. `has_recent_release_flag` is set when either an archive release event or a package publication falls in the trailing year, so the synthetic publication forces it to 1 for every row of the four 2023 cohorts. From 2024-01-01 onward it is driven by archive release events alone and does vary between repositories, taking value 0 on 21,169 rows overall. It is informative in the later cohorts and uninformative in the earlier ones, and is therefore not counted among the ten.

One consequence matters for the time-ordered split. Because `package_age_days` increases monotonically with the cohort, its test-set value of 821 lies outside the range seen during training, whose maximum is 639. A tree routes such a value to the branch of its largest training split, and a linear model extrapolates a trend that carries no repository-level meaning.³

The reported ranking metrics are unaffected. A feature with no variance cannot split a tree, so the gradient-boosted models assign it zero gain and ignore it, and a constant column standardizes to zero for the linear and neural models; all eight accordingly carry a fitted coefficient of exactly zero in the staged logistic-regression artifact, so they are inert rather than misleading. The two cohort-dependent inputs are constant *within* the test cohort, since every test row shares the observation date 2025-04-01, so they cannot reorder test rows and cannot change ROC-AUC. They could in principle shift all test probabilities by a common amount, which would show in Brier score and expected calibration error; this is not separately quantified. Two things do follow. First, the counts “60” and “17” should be read as the size of the implemented vector, not as the number of signals the reported evaluation actually had available. Second, it explains an otherwise puzzling absence in the feature attribution of Section 6.3.1: no package- or ecosystem-derived variable appears among the influential features, not because ecosystem is uninformative in general, but because this dataset contains no variation in it to learn from. Testing whether ecosystem carries signal would require a package-first sampling frame and is left to future work (Chapter 9).

A.6 System Components and Technology Selection

This section gives the component inventory and language choices. The following sections document the build and training pipeline (Appendix A.7), runtime services (Appendix A.8), and evaluation instruments (Appendix A.9). Table A.6 summarizes the components from Section 5.1.

The components use different programming languages, each selected for the part of the system

³Claude Opus 5/11 — see the Declaration on the Use of Artificial Intelligence, p. V.

Table A.6: Implemented system components

Component	Technology	Responsibility
Scoring and training workspace	Python	Historical dataset builder, feature engineering, model training, model artifacts, and the runtime scoring service.
Backend API	Go	Analysis orchestration, provider enrichment, durable job processing, and model-artifact scoring calls.
Analyst frontend	TypeScript / Next.js	Submission, repository overview, risk views, and machine-learning evaluation dashboards.
Shared schemas	TypeScript	Reusable data contracts shared by the frontend.
Orchestration scripts	Node.js	Seed generation, dataset building, notebook execution, and artifact staging commands.
Deployment assets	Docker / Kubernetes	Container images, Compose, and the Argo CD-managed Kubernetes deployment.

where it is most appropriate.

Python is used for all data engineering and machine learning, including the historical dataset builder, the feature and label computation, the model implementations, and the scoring service. Python was chosen because the libraries this thesis depends on — `numpy` for the from-scratch model implementations and `xgboost` for the gradient-boosted trees — are native to its ecosystem, and because it remains among the most widely used languages for data work [37], which eases reproduction of the analytical core. The scoring service is implemented with FastAPI.

Go is used for the backend orchestration service. The backend is primarily a concurrent input/output workload: it calls several external providers and processes durable jobs. Go’s static typing, its straightforward concurrency model, and its suitability for long-running network services make it a good fit for this role.

TypeScript with the Next.js framework is used for the analyst frontend, and TypeScript is also used for the shared data contracts. TypeScript provides type safety for the user interface and shares type definitions with the backend contracts. The frontend additionally relies on the broader JavaScript and Node.js tooling ecosystem, which is among the most widely used on GitHub [38]. This is a tooling choice rather than a statement about the ecosystems the artifact scores: the dataset seed and the demonstrator in Chapter 7 are Python-centric, and the model itself is ecosystem-agnostic.

Node.js additionally hosts the orchestration command-line scripts under `scripts/ml`. These scripts coordinate the higher-level workflow steps, such as generating the repository seed, building the dataset, executing the training notebook, and staging artifacts. They wrap the underlying Python pipeline so that the full reproducible workflow can be triggered through

consistent npm commands.

A.7 Dataset Builder and Training Pipeline

This section documents the offline path from repository seed to evaluated model files: module layout, quality reporting, feature revision, model configuration, artifact format, notebook orchestration, and the staging gate.

A.7.1 Dataset-BUILDER Modules

This subsection expands the dataset builder of Section 5.2.

The historical dataset builder is divided into focused modules, summarized in Table A.7, so that ingestion, event parsing, feature computation, and label computation can be tested independently.

Table A.7: Dataset builder modules

Module	Responsibility
adapters	deps.dev, GitHub, and npm/PyPI registry adapters used for package-to-repository resolution and stable metadata.
events	GH Archive parsing into normalized repository histories of commits, issues, pull requests, releases, stars, and forks.
features	Observation-time feature computation from history before t .
labels	12-month maintenance and inactivity label computation from the future window.
sampling	Candidate sampling and popularity-tier derivation.
pipeline	The DatasetBuilder orchestrating ingestion, snapshots, labels, and export.
storage	JSON and JSON Lines reading and writing of the intermediate artifacts.
cli	The command-line entry point used by the orchestration scripts.

The DatasetBuilder class executes the construction workflow described in the methodology. The `ingest_candidates` step loads and de-duplicates seed candidates, resolves each package to a GitHub repository, and writes the `repositories.jsonl`, `packages.jsonl`, and `package_repository_links.jsonl` files. Candidates that cannot be mapped to a GitHub repository, and forks when forks are excluded, are dropped at this stage. The `build_snapshots` step iterates over quarterly observation dates and, for each repository, constructs an `ObservationSnapshot` with explicit feature and label windows before computing the observation-time feature row. The `build_labels` step computes the future-window label for every snapshot, and `export_training_dataset` assembles the final training rows and the runtime repository feature cache.

A.7.2 Dataset-Quality Reporting

To make the per-run documentation items required by the methodology (Section 4.6) reproducible rather than transcribed by hand, a small `dataset_quality` module computes a structured quality report from the exported snapshot rows. It produces three views. The first is row accounting: the total, labeled, and unlabeled counts, the inactive and active label balance, the number of already-archived-at-observation rows that are excluded from fitting, and the completeness signals introduced above, including the `repo_silent_before_horizon` count that quantifies how many labeled examples the corrected coverage gate retains. The second is label balance per subgroup, broken down by ecosystem and by popularity tier, which exposes where labeled examples are concentrated. The third is per-feature missingness, derived from the missing-signal annotations already attached during feature extraction, so that a feature is reported as missing only when its underlying observation-time signal was unavailable rather than genuinely zero. The report is rendered in the training notebook alongside the existing coverage summary, and the row accounting deliberately records whether label-completeness signals are present in the export so that a value of zero is not misread as an absence of incomplete windows when an older export simply did not carry them.

A.7.3 Feature-Set Refinement After the Formative Review

This subsection gives the details of the first design-cycle refinement summarized in Section 5.2.

The feature set used for training was refined iteratively, as described in the methodology’s formative development step (Section 4.2). The first feature set contained variables that turned out to be poor point-in-time predictors. A representative example is a raw open-issue count: large, actively maintained projects can carry very large issue backlogs, so a model can wrongly associate a high absolute issue count with risk, even though the project is healthy. The revised feature set therefore replaces raw counts with relational and trend-based variables, such as issue closure ratio, issue backlog growth, stale-issue share, and pull-request response and merge latency, which are more comparable across projects of different sizes.

A second class of problems came from repository identity. Some packages point to mirror repositories rather than to the upstream development repository, which can make a project look inactive even though active development happens elsewhere. The mapping logic therefore prefers explicit seed and `deps.dev` mappings and resolves the primary package-to-repository link per repository, rather than treating every registry entry as an independent signal. Finally, the revised feature set keeps `repo_archived_at_obs` only as diagnostic metadata and excludes already-archived observation rows during model fitting, so that the visible archival flag cannot become a shortcut label.

A.7.4 Model Configurations and Hyperparameters

This subsection expands the model families introduced in Section 5.3 and specified in Section 4.10.

The Logistic Regression model is implemented directly in Python. It standardizes each input feature using training-set means and standard deviations, stores these standardization parameters with the model, and fits the coefficients by gradient descent on the class-weighted log-likelihood. Class-balanced sample weights counter the label imbalance, and an L2 penalty regularizes the coefficients. Because the model stores explicit per-feature coefficients, it doubles as an interpretable baseline whose weights can be inspected to check whether the engineered features behave as expected. This interpretability is what made it useful during the feature-set refinement described in Appendix A.7.3.

The neural network is a multilayer perceptron implemented on top of numpy, without a deep-learning framework. It uses two hidden layers of 64 and 32 units with ReLU activations and a sigmoid output, standardizes its inputs in the same way as the Logistic Regression model, and initializes its weights with Xavier-uniform values under a fixed random seed for reproducibility. Training uses mini-batch stochastic gradient descent on the binary cross-entropy objective, with balanced class weights and L2 regularization. The default configuration matches the methodology: a learning rate of 0.01, 150 epochs, a batch size of 256, and an L2 penalty of 0.001. The forward and backward passes are written explicitly, which keeps the model transparent and removes any heavy training dependency from the artifact.

The XGBoost model wraps the xgboost library. It trains a gradient-boosted tree ensemble with a binary-logistic objective and the same class-balanced sample weights as the other models. The default configuration uses 80 boosting rounds, a maximum tree depth of three, a learning rate of 0.08, row and column subsampling of 0.9, L2 leaf regularization, the histogram tree method, and a fixed seed. After training, the trained booster is serialized to its JSON representation and stored inside the model artifact, and the gain-based feature importances are computed and normalized so they can be displayed in the evaluation dashboard.

A.7.5 Artifact Serialization

This subsection expands the artifact format referred to in Section 5.3.

Each trained model is serialized to a self-describing JSON artifact. The artifact stores the model family and version, the ordered feature names, the model parameters (coefficients and standardization for Logistic Regression, weights and biases for the neural network, or the serialized booster for XGBoost), the calibration bins, the decision threshold, the feature-set version, and the training timestamp. Because the feature names and standardization are part of

the artifact, the runtime service can construct the feature vector in the correct order and apply the same transformations without sharing code paths with the training process. Each training run is written to a run file under the training runs directory together with the dataset hash and the held-out metrics, and a `latest-run.json` pointer records the most recently produced run.

A.7.6 Training Orchestration

This subsection expands the training workflow of Section 5.4.

The headless command `npm run ml:train` executes the training notebook non-interactively and writes the run artifacts and the latest-run pointer, so that the same notebook is used for both interactive exploration and reproducible artifact generation. The end-to-end command `npm run ml:bootstrap` chains dataset construction and notebook-based training, and the foundation variants of these commands use the preserved 5,000-repository seed and the filtered GH Archive cache for the thesis-grade run. The orchestration scripts wrap the Python pipeline rather than reimplementing it; they pass through the seed file, the GH Archive source, the output directories, the observation window, and the dataset-size guardrails, which keeps the reproducibility parameters explicit at the command line.

Listing A.1: Headless notebook-based training

```
1 npm run ml:train
```

A.7.7 Artifact Staging and Promotion Gate

This subsection expands the promotion gate summarized in Section 5.4.

Trained artifacts are not deployed automatically. The staging command `npm run ml:stage-training` validates a candidate training bundle and promotes it into the deployed bundle under `deployment/training` only if it passes a set of quality gates. This gate is the safety boundary between experimentation and deployment.

The staging script first validates the dataset itself: the snapshots must be non-empty, every labeled snapshot must come from a real GitHub repository, and the bundle must contain at least a configured minimum number of distinct repositories and distinct inactive repositories. It then validates the model run artifacts: all required model artifacts must be present and complete for the latest dataset hash, and their feature versions must match the expected full-history and cold-start feature sets. This prevents a partial or inconsistent bundle, for example one missing the cold-start models, from reaching deployment.

The most important gate is the regression comparison. Before promotion, the candidate artifacts are compared against the currently deployed baseline. For each required model, the script checks

that ROC-AUC has not dropped, and that the Brier score has not increased, by more than a configured tolerance. If any required model fails this comparison, promotion is rejected and the existing staged bundle is retained. When promotion succeeds, the script normalizes the snapshot file, the run artifacts, and the repository feature cache to their runtime paths and writes them into the deployed bundle.

A.8 Runtime Services and Deployment

This section documents the components that consume the promoted model bundle at request time — the scoring service, the backend that orchestrates an analysis, the analyst frontend, and the container and cluster deployment. None of them is part of the evidential argument of this thesis; no reported result depends on them.

A.8.1 Scoring Service

This subsection expands the scoring component of Section 5.1, and implements the undefined-feature rules of Section 4.7.

The scoring service is a FastAPI application under `mltraining/scoring/app`. It exposes a small internal interface: health and readiness probes, a `/features/extract` endpoint for feature extraction, and the `/score/model` endpoint that scores a batch of dependencies with a supplied model artifact. The service does not own a model registry; instead, the backend passes the staged artifact in the scoring request, which keeps the scoring service stateless and makes the deployed model an explicit input rather than hidden state.

For each dependency, the service builds the feature vector in the artifact’s feature order, imputing undefined signals from the training-set medians stored with the artifact (Section 4.7) and recording which signals were missing. It deserializes the artifact according to its model family, computes the raw probability, and applies the stored calibrator to obtain the calibrated inactivity probability. This probability is scaled to a 0–100 inactivity-risk score, and a complementary 12-month maintenance-outlook score is derived as its inverse. The service assigns a discrete risk bucket and an action level from the score, where the action level is one of `monitor`, `review`, or `replace_candidate`, so that the continuous score maps onto the triage actions used in the demonstrator.

The service keeps the inactivity prediction and the security posture strictly separate, exactly as required by the methodology. The OpenSSF Scorecard signal is summarized into a parallel security-posture score and is never fed into the predictive feature vector [19].

Each score is accompanied by an *evidence-support* value $s \in [0, 1]$. It is deliberately not a

statistical confidence interval or a probability that the prediction is correct: the artifact ships no coefficient covariance, so no closed-form predictive variance is available. Instead, s is an operational measure of how much observable evidence backs the individual score. It is the geometric mean of three per-repository quantities:

$$s = (c \cdot d \cdot b)^{1/3},$$

Here, c is signal coverage: the share of expected maintenance signals observed before imputation, or the model's `signal_completeness` feature when present. The in-distribution share d is the fraction of observed evidential features within 2σ of the training mean. Finally, $b = n/(n + 30)$ maps the number of validation samples n backing the matched calibration bin onto $[0, 1]$. The geometric mean makes a single weak component collapse the result. The decision margin to the threshold is reported separately and is not part of s . Explicit caveats are attached when calibration bins are missing, when the artifact's feature version is unexpected, or when input signals had to be imputed. Finally, the service returns human-readable explanation factors and evidence items so that an analyst can see which signals shaped the score rather than only the number itself.

A.8.2 Backend Orchestration Service

This subsection expands the backend component of Section 5.1 and realizes the two prediction regimes of Section 4.8.

The backend is a Go service under `backend/api`. It exposes the public REST API for analyses, repositories, jobs, and the training dataset and run endpoints. Its central responsibility is to turn a submission into an enriched, scored analysis reliably, even in the presence of slow or failing external providers.

Analyses are processed through a durable job queue backed by PostgreSQL. When a submission arrives, the service creates an analysis record and a job record in a single transaction and returns immediately. A background worker leases the next pending job, processes it, and persists the result. If processing fails, the job is retried with a bounded number of attempts and a retry delay, and it is only marked as failed once the attempts are exhausted. This design makes the analysis lifecycle resilient and observable, and it decouples the user-facing request from the slower enrichment and scoring work. For repository submissions, the service also reuses a recent completed analysis when one exists for the same repository and the same staged model set, which avoids redundant re-enrichment.

During processing, the worker enriches each repository and scores it. Enrichment fetches stable repository metadata from GitHub and retrieves the OpenSSF Scorecard signal, attaching the raw signals to the repository for display. Each enrichment provider is implemented as a separate adapter, so a single failing provider degrades the available signals rather than the whole analysis.

The backend implements the two prediction regimes from the methodology at runtime. When it enriches a repository, it attempts to look the repository up in the staged repository feature cache. On a cache hit, the repository is scored in the full-history regime using the cached observation-time features; on a miss, it falls back to the cold-start regime using only approximate features derived from live metadata, and a caveat records this. When several model artifacts are staged for the same dataset hash and regime, the backend scores the repository with each and combines the results into an ensemble profile by averaging the scores, while also retaining the per-model outputs and adding a model-agreement explanation factor that reports the spread between models.

A.8.3 Analyst Frontend

This subsection expands the frontend component of Section 5.1; the analyst workflow it serves is demonstrated in Chapter 7.

The frontend is a Next.js application under `frontend/web` that uses the React component model and the application router. It provides the analyst-facing workflow and the evaluation dashboards used during the thesis. Submission is handled through a form that accepts one or more repository URLs or a demo analysis. Once an analysis completes, the dashboard presents the repository overview with risk distribution and filtering, and a dedicated view renders the per-repository detail card with its risk profile, caveats, and evidence.

A second area of the frontend is the machine-learning evaluation surface. These pages read the training dataset and run endpoints exposed by the backend and visualize them with purpose-built charts, including a calibration curve, a model-metric comparison across the trained models, a Logistic Regression coefficient view, and a training-metric history. These views were used directly to inspect and report model behavior during the thesis.

A.8.4 Container and Cluster Deployment

This subsection expands the deployment path of Section 5.4.

The artifact is containerized for both local and production-style operation. Canonical image definitions live under `deployment/docker`, and a Docker Compose file orchestrates PostgreSQL, the Go backend, the scoring service, and the frontend for local development; a single `npm run dev` command builds and starts the full stack and waits for the services to become healthy. For production-style operation, the project is deployed to a k3s cluster managed by Argo CD, with container images published to a registry and the application served behind a public domain [29]. Because the deployed model bundle is a versioned set of files under `deployment/training`, deployment and model promotion remain auditable: the running service uses exactly the artifacts

that passed the staging gate described in Appendix A.7.7.

A.9 Evaluation Instruments

This section gives the mechanics of the two evaluation instruments introduced in Section 5.3. They are of different kinds, and the distinction matters for what each result may claim. The slice evaluation is a re-analysis: it re-scores the predictions of the same fitted model within subgroups. The repository-disjoint split is a separate training run under a different partitioning rule, so it produces its own fitted model. Neither alters the reported run, whose serialized artifacts and headline metrics are unchanged, because the reported configuration retains the chronological split.

A.9.1 Subgroup Definitions for the Slice Evaluation

The resulting figures are reported in Section 6.3.3 and tabulated in Appendix A.15.

A single aggregate metric can hide a subgroup on which the model performs poorly, because a strong majority subgroup can mask a weak minority one. A slice-evaluation helper therefore re-scores the same held-out predictions within subgroups, reusing the existing metric implementation so that no separate evaluation logic is introduced. To make this possible the training pipeline also returns the per-row held-out predictions alongside the aggregate result; these are used only for analysis and are not written into the model artifact, so the serialized artifacts and the reported headline metrics are unchanged. The harness includes a consistency check that requires the aggregate over all slices to reproduce the model’s reported held-out ROC-AUC, so the per-slice figures are guaranteed to originate from the same execution as the headline metrics.

The subgroups are those that vary meaningfully in the foundation dataset: popularity tier, a cold-start-like versus has-history regime, and whether the repository was still active at the observation date. The last of these is the active-at- t subset referenced in the evaluation strategy (Section 4.11) and in the discussion of activity persistence in Chapter 8: restricting the evaluation to repositories that are still active at t separates genuine early-warning skill from the trivial tendency of already-quiet repositories to remain quiet. Two candidate dimensions are deliberately excluded because the sampling frame holds them constant. Ecosystem is excluded because the repository-first foundation seed assigns the same provenance ecosystem to every row; cross-ecosystem evaluation would require a package-first sampling frame and is left to future work. Repository-age band is excluded because the offline build carries no repository creation date, so `repo_age_days` is identically zero and every row would fall into a single band (Section 4.4); reporting such a slice would suggest an age analysis that the data cannot support. Because slicing reduces the sample size, each slice is reported with its own row count, ranking metrics that are

undefined for single-class slices are reported as absent rather than as a misleading value, and thin slices are interpreted with corresponding caution.

A.9.2 Mechanics of the Repository-Disjoint Split

This subsection addresses the repository-recurrence threat of Chapter 8 that arises from the split described in Section 4.6.

The reported evaluation uses a chronological split: rows are ordered by observation date and cut into training, validation, and test partitions. Because observations are taken quarterly, a repository contributes several snapshots over time, so the same repository can appear in both the training and the test partition, and the twelve-month label windows of its successive snapshots overlap. The held-out metric then partly reflects generalization to later observations of repositories already seen during training, which is optimistic relative to performance on genuinely new repositories. This is the deployment-realistic question — the system does re-score repositories over time — but it should not be the only question asked.

A further evaluation iteration therefore adds a repository-disjoint split that assigns every snapshot of a repository to a single partition, so that the held-out set contains only repositories absent from training. The split is selectable through a configuration option whose default preserves the chronological behavior, so the reported artifacts and headline metrics are unchanged, and the pipeline asserts that the partitions are repository-disjoint whenever the grouped split is used.

Because grouping changes which rows are available for fitting, this is a full second training run rather than a re-scoring of the reported model. The same pipeline entry point is invoked with the grouped strategy, and it refits in the same order as the reported run: the feature preprocessing, then the classifier on the grouped training partition, then the histogram calibrator and the F1-selected decision threshold on the grouped validation partition. The grouped test partition enters only after those decisions are fixed. The two runs therefore report two different fitted models, and the difference between their held-out ROC-AUC values bounds the optimism introduced by repository recurrence. In practice the held-out ROC-AUC decreases only marginally, which indicates that the model’s discrimination does not rest on memorizing individual repositories. The comparison is between held-out repositories drawn from the same retrospectively sampled foundation frame (Section 4.4), so it does not establish transfer to open-source repositories in general. The exact figures for both splits are produced in the training notebook and reported in Chapter 6.

A.10 Dataset Construction Steps

This section expands the dataset construction workflow of Section 4.5; its software realization is described in Section 5.2.

The historical dataset builder creates its dataset through the sequence of intermediate steps summarized in Table A.8, writing the intermediate files listed below under the configured output directory so that the construction process is auditable and the generated dataset can be inspected before training.

Table A.8: Historical dataset construction workflow

Step	Description
1	Load repository candidates from the foundation seed.
2	Use the seed buckets to obtain a mixture of active, dormant, and archived repository histories.
3	Resolve repository identities from explicit seed fields or package metadata where required.
4	Enrich stable repository metadata through GitHub.
5	Parse GH Archive event files into normalized repository histories.
6	Build quarterly observation snapshots for each repository.
7	Compute observation-time features using only data available at or before t .
8	Compute 12-month labels using only the future window $(t, t + 12 \text{ months}]$.
9	Export the final snapshot dataset for the existing model-training workflow.
10	Write a repository feature cache for runtime full-history scoring.

The documented intermediate outputs are `repositories.jsonl`, `packages.jsonl`, `package_repository_links.jsonl`, `observation_snapshots.jsonl`, `snapshot_features.jsonl`, `snapshot_labels.jsonl`, `training_snapshots.json`, and `repository-feature-cache.json`.

A.11 Evaluation Metric Definitions

This section gives the formal definitions of the metrics named in the evaluation strategy (Section 4.11) and reported throughout Chapter 6.

All metrics are computed on the calibrated probabilities of the held-out test set. Let N be the number of evaluation samples, $p_i \in [0, 1]$ the predicted probability of inactivity for sample i , and $y_i \in \{0, 1\}$ the true label (1 = inactive within twelve months). The positive (inactive) base rate of the test set is

$$\bar{p} = \frac{1}{N} \sum_{i=1}^N y_i, \quad (\text{A.1})$$

reported as the *inactive 12m rate*.

Ranking. ROC-AUC (equivalently AUROC, the notation used in the equation below) is computed as the Mann–Whitney statistic over all positive–negative pairs, that is, the probability that a random inactive repository is scored above a random active one, with ties counted as one half:

$$\text{AUROC} = \frac{1}{N_+ N_-} \sum_{i: y_i=1} \sum_{j: y_j=0} \left(\mathbb{I}[p_i > p_j] + \frac{1}{2} \mathbb{I}[p_i = p_j] \right), \quad (\text{A.2})$$

where N_+ and N_- are the numbers of positive and negative samples. It is threshold- and calibration-independent: 0.5 is random ranking and 1.0 is a perfect ranking.

Ranking under class imbalance. Average precision summarizes the precision-recall curve. Samples are sorted by decreasing score; at each distinct score threshold n , let P_n and R_n be the precision and recall attained by predicting positive down to that threshold. Average precision is the step-wise sum

$$\text{AP} = \sum_n (R_n - R_{n-1}) P_n, \quad R_0 = 0, \quad (\text{A.3})$$

with tied scores collapsed into a single threshold so that a signal with many ties is neither rewarded nor penalized for them. No interpolation between points is applied. Unlike ROC-AUC, average precision ignores true negatives and so does not credit a model for correctly ranking a large negative class. Its no-skill reference is not 0.5 but the positive base rate $\bar{p}$, which a random ranking attains, so an AP value is interpretable only alongside the prevalence of the set it was computed on. It is reported for the early-warning subset in Section 6.1.3.

Probabilistic accuracy. The Brier score is the mean squared error between probability and outcome, and the log loss is the binary cross-entropy (with a small ϵ for numerical stability):

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2, \quad (\text{A.4})$$

$$\text{LogLoss} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right]. \quad (\text{A.5})$$

Both are lower-is-better and penalize miscalibration; the log loss grows without bound as a confident prediction approaches the wrong outcome, so it punishes overconfidence more sharply than the bounded Brier score.

Calibration. The expected calibration error partitions $[0, 1]$ into $B = 10$ equal-width bins. For bin b with member set $\mathcal{B}_b$, let conf_b be the mean predicted probability and acc_b the empirical positive rate; the ECE is the sample-weighted average gap between predicted confidence and

observed frequency:

$$\text{ECE} = \sum_{b=1}^B \frac{|\mathcal{B}_b|}{N} |\text{conf}_b - \text{acc}_b|. \quad (\text{A.6})$$

Thresholded classification. A decision threshold τ maps probabilities to decisions $\hat{d}_i = \mathbb{1}[p_i \geq \tau]$, yielding the counts of true and false positives and negatives (TP, FP, FN, TN). From these,

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{F1} = \frac{2 \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (\text{A.7})$$

The F1 score is the harmonic mean of precision and recall, so it cannot be made large by optimizing one at the expense of the other; τ is selected on the validation set to maximize it.

Combined quality score. The quality score converts ranking and probabilistic accuracy into skill scores relative to the trivial base-rate predictor and averages them, weighting ranking at 0.6 and calibration at 0.4. With the climatological Brier score $\bar{p}(1 - \bar{p})$ and $\text{clip}_{[0,1]}$ clamping to the unit interval,

$$s_{\text{AUC}} = \text{clip}_{[0,1]} \left(\frac{\text{AUROC} - 0.5}{0.5} \right), \quad (\text{A.8})$$

$$s_{\text{Brier}} = \text{clip}_{[0,1]} \left(1 - \frac{\text{Brier}}{\bar{p}(1 - \bar{p})} \right), \quad (\text{A.9})$$

$$\text{Quality} = \text{clip}_{[0,1]} (0.6 s_{\text{AUC}} + 0.4 s_{\text{Brier}}). \quad (\text{A.10})$$

A value of 0 indicates no improvement over predicting the base rate and 1 indicates perfect ranking and calibration. Because both skill terms are measured against the test-set base rate $\bar{p}$, the quality score is conditional on the sampled prevalence and is intended for model comparison rather than as a standalone performance guarantee.

A.12 Reproducibility

This section makes the dataset construction workflow of Section 4.5 reproducible, and records the identifying metadata of the run whose results are reported in Chapter 6.

The dataset builder and training pipeline are executed through repository scripts. The foundation seed can be generated with:

```
npm run ml:seed:foundation -- '
  --target-repositories 5000 '
  --github-token <TOKEN>
```

The larger foundation dataset can then be built and trained with:

```
npm run ml:bootstrap:foundation -- '
  --github-token <TOKEN> '
  --gharchive-source <GHARCHIVE_PATH>
```

The underlying full command path uses a generated foundation seed, a GH Archive source directory, a dedicated training output directory, a training snapshot export path, and a repository feature cache output path. The repository documentation recommends the foundation path for the thesis/training base, while the smaller default bootstrap path remains a local smoke-test workflow [31].

For each final training run, the following items should be documented:

- seed generation command and seed metadata,
- GH Archive coverage period,
- output directory,
- training snapshot path,
- repository feature cache path,
- dataset hash,
- model artifact path,
- model name and version,
- feature version,
- number of total, labeled, and unlabeled rows,
- train, validation, and test split sizes,
- held-out evaluation metrics.

The reported full-history run uses dataset hash:

```
2d29a928dac34ea1dab919f87e89386f2e6eac3bda59aee7a625c5ad0873b0d8
```

and exports the model artifact:

```
deployment/training/runs/
20260709T205727480957Z-xgboost-full-history-2d29a928dac3.json
```

The reported held-out metrics are those recorded in this staged run file, which is the authoritative record of the reported experiment. The corresponding source revision — containing this dataset, the staged run artifacts, and the training notebook — is archived at commit `f5e688e` [29], so that the reported run can be traced to a fixed state of the dataset builder and training pipeline. One caveat applies to the hash above: it is the value recorded by the staged run at training time, whereas the dataset file committed at `f5e688e` was re-serialized afterwards and therefore carries a different content hash (`f71b9f7a...`). Re-training `xgboost-full-history` on the committed file reproduces the recorded held-out metrics exactly — ROC-AUC 0.957852, Brier 0.077939, expected calibration error 0.035155, F1 0.916369, and the selected threshold 0.506689, each identical to six decimal places — so the two files differ only in serialization: the recorded hash identifies the run, and the committed file reproduces it.⁴

A.13 Label-Definition Sensitivity

Table A.9 reports the robustness check of Section 6.4: the held-out evaluation repeated under one-of-four, two-of-four (primary), and three-of-four variants of the maintenance rule. All 50,000 snapshots are re-labeled from the same stored future-window signals, and the full-history XGBoost model is re-trained per variant on the identical time-aware split. Because the recency baseline is undefined for the 444 test rows without an observable commit history (Section 6.1.1), the model is reported both on the full test set and re-scored on the 4,556 recency-defined rows, so that the model and the two trivial rules are compared on identical rows. Two checks confirm that the re-labeling harness matches the primary pipeline: the two-of-four labels derived from the stored signals agree with the exported `label_inactive_12m` on all 50,000 snapshots, and the two-of-four row reproduces the reported headline figures. The analysis is reproduced by `scripts/ml/label_sensitivity.py`, which performs both checks and aborts if the first one fails (Appendix A.1).

⁴Claude Opus 5/9 — see the Declaration on the Use of Artificial Intelligence, p. V.

Table A.9: Sensitivity of the held-out evaluation to the maintenance-rule threshold. “Inactive rate” is the positive-class share of the test set. “XGBoost (all)” is the full-history model on all 5,000 test rows; the remaining columns are on the 4,556 recency-defined rows, where model and baselines are scored on identical rows. “Margin” is the model there minus the stronger of the two trivial rules, which is the recency rule under the permissive variant and commit volume under the other two.

Maintenance rule	Inactive rate	XGBoost (all)	XGBoost (rec.)	Recency	Commits	Margin
1-of-4 (permissive)	0.581	0.961	0.975	0.965	0.960	+0.010
2-of-4 (primary)	0.661	0.958	0.974	0.957	0.958	+0.016
3-of-4 (strict)	0.746	0.959	0.972	0.940	0.949	+0.023

A.14 Full Demonstrator Ranking

Table A.10 lists the complete ranked inactivity-risk view for all 30 dependency repositories of ArchipelagoMW/Archipelago produced by the model in the demonstrator (Chapter 7); the body reports the flagged and medium-risk subset in Table 7.1.

The explanation factors of the highest-ranked repository, `bsdifff4 (ilanschnell/bsdifff4)`, risk 92.1; Section 7.2), taken directly from the scored output, illustrate the intended semantics: the two cold-start ensemble members predict 97.0% and 87.2% inactivity risk, the model-agreement factor reports the spread between them, and the input-completeness factor records that 75% of expected signals were available before imputation. The moderate evidence support of 0.69 and the caveats attached to the score — that it came from the cold-start model, no full-history cache row being available, and that two release-related signals were imputed — are what present the flag as a prompt for manual review rather than a definitive judgement.

Table A.10: Complete ranked inactivity-risk view for the dependency repositories of ArchipelagoMW/Archipelago, as produced by the model. Risk is the 0–100 inactivity-risk score; “Supp.” is the per-prediction evidence support; “Sec.” is the parallel 0–100 security-posture score derived from the repository’s OpenSSF Scorecard record, which is not an input to the inactivity model (Section 4.1) and is shown as — for the three repositories with no available Scorecard record; “Regime” is full-history (FH) or cold-start (CS).

Package	Source repository	Risk	Action	Supp.	Sec.	Regime
bsdiff4	ilanschnell/bsdiff4	92.1	replace_candidate	0.69	6	CS
setproctitle	dvarrazzo/py-setproctitle	80.4	replace_candidate	0.61	11	CS
jinja2	pallets/jinja	77.1	review	0.77	58	CS
cymem	explosion/cymem	74.9	review	0.75	24	CS
markupsafe	pallets/markupsafe	74.2	review	0.75	54	CS
pymem	srounet/Pymem	54.3	review	0.47	—	CS
flask-compress	colour-science/flask-compress	48.5	monitor	0.45	13	CS
pony	ponyorm/pony	42.8	monitor	0.59	1	CS
flask-limiter	alisaifee/flask-limiter	41.9	monitor	0.58	18	CS
kivymd	kivymd/KivyMD	41.8	monitor	0.47	15	CS
waitress	Pylons/waitress	41.3	monitor	0.59	30	CS
schema	keleshev/schema	37.3	monitor	0.78	23	CS
pyyaml	yaml/pyyaml	35.4	monitor	0.77	43	CS
orjson	ijl/orjson	31.1	monitor	0.61	27	CS
colorama	tartley/colorama	27.8	monitor	0.54	30	CS
pyshortcuts	newville/pyshortcuts	25.0	monitor	0.75	11	CS
werkzeug	pallets/werkzeug	24.3	monitor	0.76	58	CS
certifi	certifi/python-certifi	23.5	monitor	0.57	59	CS
platformdirs	tox-dev/platformdirs	22.6	monitor	0.77	—	CS
pathspec	cpburnz/python-pathspec	16.6	monitor	0.83	—	CS
kivy	kivy/kivy	15.0	monitor	0.85	36	CS
websockets	python-websockets/websockets	10.3	monitor	0.90	52	CS
flask	pallets/flask	9.5	monitor	0.91	77	FH
flask-cors	corydolphin/flask-cors	8.9	monitor	0.91	46	CS
typing-extensions	python/typing_extensions	5.8	monitor	0.95	99	CS
bokeh	bokeh/bokeh	5.3	monitor	0.71	65	CS
flask-caching	pallets-eco/flask-caching	5.0	monitor	0.95	49	CS
mistune	lepture/mistune	4.2	monitor	0.96	43	CS
psutil	giampaolo/psutil	3.4	monitor	0.72	47	CS
cython	cython/cython	2.6	monitor	0.98	31	CS

A.15 Per-Subgroup Metrics of the Slice Evaluation

This section gives the full per-subgroup breakdown of `xgboost-full-history` whose three findings are interpreted in Section 6.3.3.

Table A.11: Held-out ROC-AUC and Brier score of `xgboost-full-history` within subgroups of the test set. “Inactive rate” is the positive rate in the subgroup.

Dimension	Subgroup	n	Inactive rate	ROC-AUC
Popularity	high	2297	0.527	0.972
Popularity	medium	1753	0.778	0.964
Popularity	low	950	0.767	0.846
Activity at t	active@ t	1794	0.225	0.905
Activity at t	quiet@ t	3206	0.905	0.911
History regime	has-history	2295	0.350	0.921
History regime	cold-start-like	2705	0.924	0.929

A.16 Ablation: The Human-Actor Filter

Commit- and contributor-based features and the inactivity label are computed from human actors only; automated (bot) accounts are excluded (Section 4.7). Because this design choice could plausibly be omitted, it was tested by rebuilding the full 5,000-repository dataset with the filter disabled through a single build flag, with every other setting identical, yielding 50,000 snapshots that align one-to-one with the filtered build. The headline outcome is summarized in Section 6.4: the filter earns its place on construct-validity grounds rather than on discrimination. Three effects were measured.

Label corruption. Disabling the filter flips 357 of 50,000 labelled snapshots (0.71%) from inactive to active, and *none* in the opposite direction. The effect is strictly one-directional: a bot commit or merge inside the future window can satisfy the two-of-four maintenance rule, so an otherwise dormant repository is relabelled as maintained, whereas bots never make an active repository look inactive. Omitting the filter would therefore bias the target by systematically *masking* inactivity, which is the error direction that matters most for a risk signal. That asymmetry is invisible to a headline metric computed on the corrupted data, which is why the filter is retained despite the small predictive effect below.

Feature inflation. On the observation-time side, bots inflate the activity features substantially: with the filter disabled, `commits_365d` changes on 7,858 of 50,000 snapshots (15.7%) by a

mean of +938 commits, though the median change is only +80, so the mean is driven by a small number of heavily automated repositories. `commits_90d` rises by a mean of +388 and `contributors_365d` by +1.4 distinct contributors, while pull-request *count* features are unaffected because they are not actor-filtered. A small number of bot-driven repositories would thus dominate the activity signal.

Predictive effect. A naive comparison, in which each model is scored against the labels of the dataset it was trained on, would show bots to be nearly harmless — but it is misleading, because the unfiltered model is then scored against a partially corrupted target. A cleaner test trains on the bot-contaminated *features* while evaluating against the clean, human-filtered *labels* (Table A.12): the contaminated features are consistently worse across all four families, though by a small margin (ROC-AUC falls by 0.0006–0.0011; Brier rises by up to 0.0010).

Table A.12: Cross-test of the human-actor filter: training on bot-contaminated features versus clean features, both evaluated against the identical clean (human-filtered) label on the test set. A lower ROC-AUC or higher Brier for the contaminated features means the filter helps.

Model	ROC-AUC		Δ ROC-AUC	Δ Brier
	clean	contam.		
logistic-regression-full-history	0.9494	0.9484	−0.0010	+0.0005
xgboost-full-history	0.9579	0.9569	−0.0010	+0.0001
logistic-regression-cold-start	0.9318	0.9308	−0.0011	+0.0005
xgboost-cold-start	0.9531	0.9524	−0.0006	+0.0010

Declaration of Authorship

I declare that I wrote this thesis independently, that I used no sources or aids other than those stated, and that all passages taken from other works, whether verbatim or in substance, are identified as such. The drawings, figures, and tables in this thesis were created by me or are provided with a corresponding source reference. The use of generative artificial intelligence is declared separately and in full in the Declaration on the Use of Artificial Intelligence (page V). This thesis has not previously been submitted, in the same or a similar form, to any examination authority.

Forst (Lausitz), 29.07.2026

Eric Krause